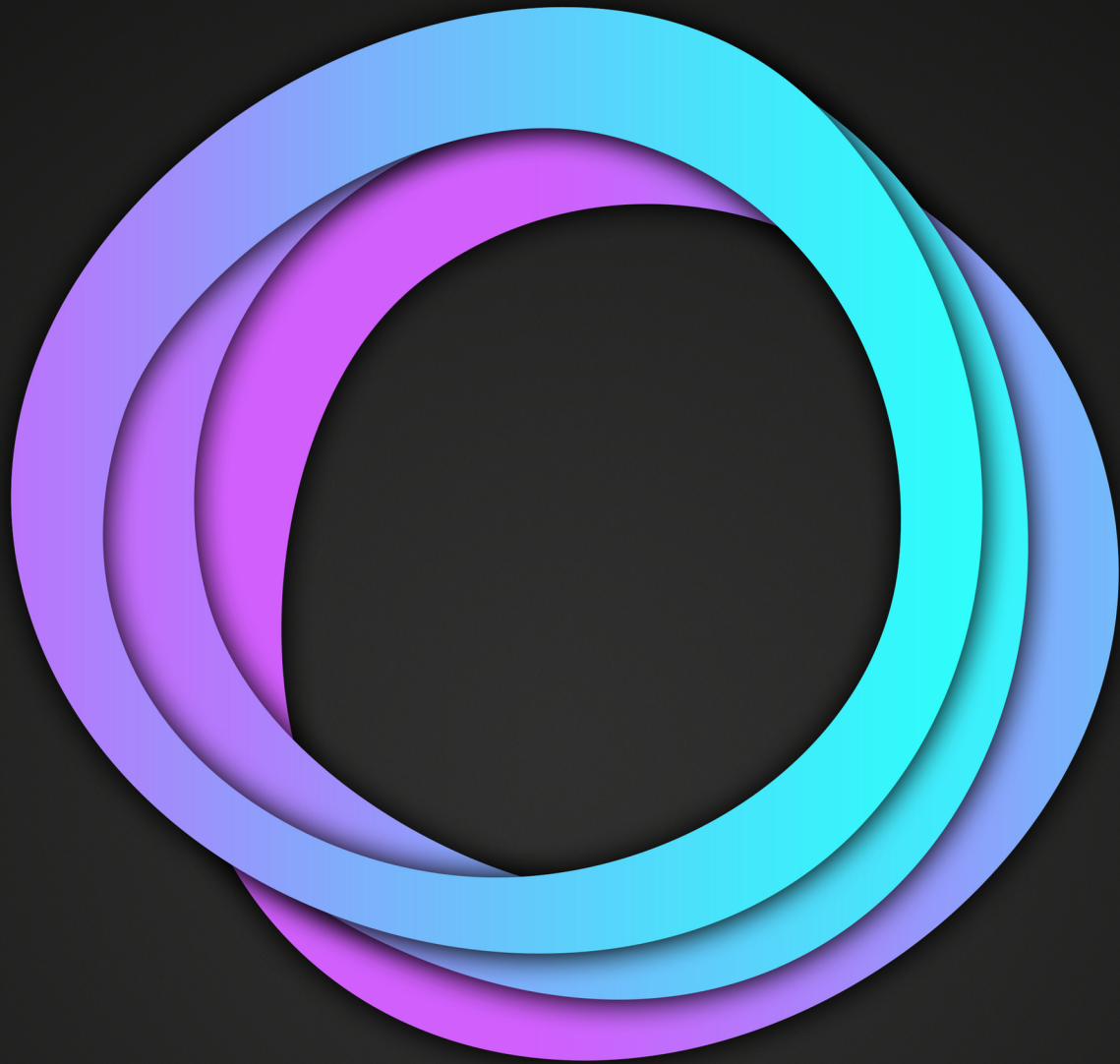


# Core Data Synchronization

with



ensembles

Drew McCormack, PhD

# Core Data Synchronization with Ensembles (BETA)

drewmccormack@mac.com

This book is for sale at <http://leanpub.com/ensembles>

This version was published on 2017-06-15



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2014 - 2017 drewmccormack@mac.com

# Contents

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>Acknowledgments</b> . . . . .                      | <b>i</b>  |
| <b>Preface</b> . . . . .                              | <b>ii</b> |
| <b>Introduction</b> . . . . .                         | <b>iv</b> |
| What is this Book About? . . . . .                    | iv        |
| What Will Not be Covered? . . . . .                   | iv        |
| <b>Introducing Ensembles</b> . . . . .                | <b>1</b>  |
| What is Ensembles? . . . . .                          | 1         |
| Design Goals . . . . .                                | 2         |
| How Does it Work? . . . . .                           | 6         |
| Ensembles <i>versus</i> The Rest . . . . .            | 10        |
| <b>Installing Ensembles</b> . . . . .                 | <b>13</b> |
| Downloading Ensembles . . . . .                       | 13        |
| Integrating Ensembles into an Xcode Project . . . . . | 14        |
| Idiomatic App . . . . .                               | 19        |
| Deploying Ensembles Server . . . . .                  | 20        |
| <b>Quick Start Guide</b> . . . . .                    | <b>24</b> |
| E.L.M. . . . .                                        | 24        |
| Global Identifiers . . . . .                          | 28        |
| Where Next? . . . . .                                 | 30        |
| <b>The Ensemble</b> . . . . .                         | <b>31</b> |
| What is an ‘Ensemble’? . . . . .                      | 31        |
| The CDEPersistentStoreEnsemble Class . . . . .        | 31        |
| Delegate Methods . . . . .                            | 35        |
| Notifications . . . . .                               | 36        |
| <b>Leeching and Deleeching</b> . . . . .              | <b>38</b> |
| Leeching . . . . .                                    | 38        |
| Initial Data Migration . . . . .                      | 39        |
| Deleeching . . . . .                                  | 41        |
| <b>Merging</b> . . . . .                              | <b>43</b> |

## CONTENTS

|                                                 |           |
|-------------------------------------------------|-----------|
| What Happens During a Merge? . . . . .          | 43        |
| Triggering Merges . . . . .                     | 45        |
| When to Trigger Merges . . . . .                | 46        |
| Updating Contexts . . . . .                     | 50        |
| Delegate Methods . . . . .                      | 51        |
| Merge Errors . . . . .                          | 53        |
| <b>Saving . . . . .</b>                         | <b>55</b> |
| Monitoring Saves . . . . .                      | 55        |
| Committing Changes . . . . .                    | 56        |
| Approaches to Saving . . . . .                  | 57        |
| Termination . . . . .                           | 59        |
| Delegating Saving . . . . .                     | 61        |
| <b>Models . . . . .</b>                         | <b>63</b> |
| Loading a Model . . . . .                       | 63        |
| Designing a Model . . . . .                     | 63        |
| Migrations . . . . .                            | 64        |
| Object Identity . . . . .                       | 64        |
| Working with Data . . . . .                     | 66        |
| Batched Traversals (Ensembles 2 only) . . . . . | 67        |
| <b>Repairing Conflicts . . . . .</b>            | <b>70</b> |
| <b>Testing and Debugging . . . . .</b>          | <b>71</b> |
| Unit Tests . . . . .                            | 71        |
| <b>Backends . . . . .</b>                       | <b>72</b> |
| iCloud Drive . . . . .                          | 72        |
| CloudKit . . . . .                              | 73        |
| Dropbox Core API . . . . .                      | 76        |
| WebDAV . . . . .                                | 81        |
| Zip Compression . . . . .                       | 82        |
| Encryption . . . . .                            | 83        |
| Ensembles Server (Node.js, S3) . . . . .        | 84        |
| <b>Adding a Custom Backend . . . . .</b>        | <b>88</b> |
| CDECloudFileSystem . . . . .                    | 88        |

# Acknowledgments

I would like to thank those in the Cocoa developer community who have contributed to sync technologies on Apple platforms, and have thereby helped me — directly or indirectly — to improve the Ensembles framework.

Milen Dzhumerov Tim Isted Marcus Zarra Christian Beer Charles Parnot Joel Grasmeyer Steve Tibbett Michael Fey Kevin Hctor Daniel Pasco Dave Verwer (iOS Dev Weekly) Chris Eidhof and Daniel Eggert (objc.io) Chris Price (iOSDevUK) Steve Scott (NSConference) Niklas Saers (GOTO Conference)

And all of the other developers who have given Ensembles a try, and provided constructive feedback.

Special thanks to Marcello Luppi (Wrinkly Pea Design) for preparing the artwork and icon of Ensembles.

# Preface

Late in 2013, I announced at [iOSDevUK](http://www.iosdevuk.com)<sup>1</sup> that I was working in my spare time on a new open source, sync framework for Core Data: Ensembles. A month after that, while rehashing the presentation at [GOTO Aarhus](http://gotocon.com/aarhus-2013/)<sup>2</sup> in Denmark, I pushed the first working source code to [GitHub](https://github.com/drewmccormack/ensembles)<sup>3</sup>.

It had been a long journey, and there was still a long way to go. I had been struggling for years to replace the Wi-Fi sync in my [Mental Case](http://www.mentalcase.com)<sup>4</sup> product with a more contemporary cloud-based solution. Mental Case deals with non-trivial amounts of data, and a moderately complex data model; the existing options for Core Data synchronization, including Apple's iCloud, had fallen frustratingly short of the mark.

Although I did eventually ship Mental Case 2 with cloud sync, built upon the [TICDS framework](https://github.com/nothirst/TICoreDataSync)<sup>5</sup>, I had never been completely happy with the result. TICDS was a brave first attempt by its developer, Tim Isted, at a solution to the cloud sync problem at a time when no other options existed. But it pre-dated even iCloud, had lost its creator to Apple, and was already starting to show its age.

I contributed code to the TICDS project, and considered whether it could be updated to incorporate design changes that we now know work better. In the end, I decided it would involve such widespread changes, it was probably more work than just starting again, with the latest Objective-C language goodness, and the knowledge garnered from TICDS, iCloud, and other attempts to solve the problem.

And so, back in April 2013, I started hacking away on the beginnings of a framework codenamed 'Syncophant'. I didn't have a clear direction: the design had not fully crystallised. After a month or two, I realised that I would never get the framework to any stable state without good tests, so I started developing each component using Xcode 5's new unit testing framework.

As I went, I developed clear ideas about how things should work in a peer-to-peer synchronization framework like Ensembles. I started to see where other frameworks were going wrong, and also where they were doing things right. I took the best, and looked for ways to avoid the worst.

In this way, I came to develop a set of informal requirements and specifications for the framework. These became the essence of the Ensembles framework, and they are covered in the introductory chapters.

In March 2014, I announced Ensembles 1.0 at [NSConference](http://nsconference.com)<sup>6</sup>. The ideas had crystalized, the tests passed, and there were already a few apps shipping with it. In short, it was ready for production.

You could argue that Ensembles is at a disadvantage with respect to the incumbent iCloud—Core Data framework, with Apple holding all the cards. It is true Apple has access to private APIs, and the credibility that comes with being Apple. Luckily, the Core Data API is complete enough to develop Ensembles without the need for private APIs, and Ensembles will hopefully establish its own form of credibility as it ships in more and more apps.

---

<sup>1</sup><http://www.iosdevuk.com>

<sup>2</sup><http://gotocon.com/aarhus-2013/>

<sup>3</sup><https://github.com/drewmccormack/ensembles>

<sup>4</sup><http://www.mentalcase.com>

<sup>5</sup><https://github.com/nothirst/TICoreDataSync>

<sup>6</sup><http://nsconference.com>

And there are also advantages that a small open source project like Ensembles has over a major company like Apple. Agility for one. Apple is restricted in how often it can issue updates to frameworks, while a bug can be fixed in Ensembles in a matter of hours, and in your app the next day.

Apple are also restricted in what they can include in their offering. You should not hold your breath for Dropbox support in Core Data, while Ensembles includes it out-of-the-box, and can support virtually any new backend with a few hours of work.

Adopting Ensembles is an investment in the future. It's a solid product, with nothing to hide, and can adapt as the market place evolves. Contrast that to opaque commercial offerings that lock you in to one service, and the decision is made palpably simpler.

# Introduction

- Book is about how to use Ensembles framework
- What is Ensembles?
- Sync for Core Data apps
- Introducing ensembles covers generally how it works, and design goals
- Covers installing and integrating
- Aspects of setting up your model for sync
- Covers more advanced topics (conflict resolution)
- Various backends

## What is this Book About?

- An open source framework for syncing Core Data stores
- Only useful on Apple systems
- Only useful if you use Core Data

## What Will Not be Covered?

- Assume understanding of Core Data



# Introducing Ensembles

Before digging into Ensembles itself, it is worth taking a peek from a distance. Establish in your head the design goals of the project, and get a general feel for how it goes about the task of data synchronization. That's what this chapter is about. What is Ensembles all about?

If you don't like to mess around, jump straight to the next chapter to sink (pun fully intended) your teeth into installing and making use of Ensembles in your apps.

## What is Ensembles?

### The name

The name 'Ensembles' may seem like an odd choice for a sync framework. It derives from the central concept upon which the framework is based: the *ensemble*.

A set of syncing persistent stores is called an *ensemble*, because it is like a group of musicians playing together. Each participant is an individual, but through an exchange of information — musical notes — they strive to form a cohesive whole.

The objective of the Ensembles framework is the same. Each persistent store stands alone, but there is an interchange with other stores to achieve conformity. This high-level organization of persistent stores is an *ensemble* of stores.

### The framework

Ensembles is a framework for backend-agnostic, peer-to-peer synchronization of Core Data persistent stores. That's a very succinct explanation, so let's break it down a bit.

*Backend agnostic* refers to the fact that Ensembles can be made to work with any server or cloud service that is capable of transferring files (eg iCloud, Dropbox, FTP, Amazon S3). In fact, it goes a bit further than that, because it is also possible to make it work with pure peer-to-peer communication such as Bluetooth or Local WiFi. For example, there is a backend included for the Multipeer Connectivity framework, which doesn't make use of any server at all.

Ensembles includes backends for major services like iCloud and Dropbox, but it is relatively easy to add a new backend for an unsupported service, or even a custom server. You just need to implement a protocol (`CDECloudFileSystem`), which includes file operations like uploading, downloading, and deleting. This typically takes a few hours, at which point Ensembles should be capable of syncing via the service.

The last term that needs some explanation is *peer-to-peer*. Didn't I just say an online service is often involved? That is true, but Ensembles places all of the burden of maintaining data consistency on the client Macs and iOS devices. Any service involved in transferring files can be seen more as being part of the data transport network — it needs no insight into the content of the files. In this sense, I like to think of the framework as being peer-to-peer, even if it isn't always strictly true.

## Ensembles 1.x

The goal in developing Ensembles was initially to develop a robust framework, but not necessary an optimal one. Ensembles 1.x can be used to sync most of the standard apps found in the App Store, but for complex models or large data sets, may not be suitable. On occasion it may load the full data store from disk, and on constrained memory devices like iPhones, this can sometimes be a problem.

The original Ensembles 1.x project is still supported and maintained. What's more, it is fully open source, and can be downloaded directly from GitHub.

The well-established [MIT](http://opensource.org/licenses/MIT)<sup>7</sup> licence has been applied. The MIT licence is commonly used for iOS and Mac open source projects. It is a very liberal licence; in short, you can take and modify the code, ship it in commercial products, submit it to the iTunes App Store, and all without having to release any source code yourself. You don't even have to release any changes you make to Ensembles.

What you do have to do is include the MIT licence somewhere on your web site or in your app. A common choice is to add it in an About box, which most apps include.

The Ensembles 1.x project on GitHub is a great way to try out Ensembles, and — in many cases — will be a perfectly good solution to your sync problem. If it isn't, there is always...

## Ensembles 2

Ensembles 2 is a newer variant of Ensembles which has been largely rewritten to dramatically reduce memory usage, increase performance, and reduce cloud storage usage. It is a drop-in replacement for Ensembles 1.x; if you have your app syncing with Ensembles 1.x, it should be very little work to adopt Ensembles 2.

Although the API for Ensembles 2 is very similar to Ensembles 1.x, the two are not binary compatible. They use different file formats, so if you are migrating your Ensembles 1.x app to Ensembles 2, you create a new Ensemble for the new framework. They should not refer to the same cloud data.

The Ensembles 2 framework is not released under an open source license; it is only available when [purchasing](#)<sup>8</sup> a support package from [The Mental Faculty B.V.](#)<sup>9</sup>. Some of the packages include full source code, but the code may not be distributed to third parties.

## Design Goals

As I developed Ensembles, I built up a set of design goals and requirements. These helped guide the project in the right direction, and it is useful to understand these goals to better appreciate how the framework works, and why it works that way.

### Requires minimal changes to existing code

There is nothing in Core Data that should necessitate major changes to your data model, or class hierarchy, in order to support synchronization. The `NSManagedObjectContext` class already fires notifications before and after each save, with all of the information necessary to form change sets.

---

<sup>7</sup><http://opensource.org/licenses/MIT>

<sup>8</sup><http://ensembles.io>

<sup>9</sup><http://mentalfaculty.com>

A major goal of Ensembles was that the API should be as simple as possible, and that it should be as non-invasive as possible. You should not need to subclass `NSManagedObjectContext` or `NSManagedObject`. You should not have to alter your model.

In stark contrast to other solutions, you also shouldn't be compelled to alter the Core Data stack (*eg* suddenly have to remove your `NSPersistentStore`) just to accommodate Ensembles. Your `NSManagedObjectContext` should proceed unhindered, even when Ensembles has no connection to the cloud, or a catastrophic problem arises, such as the user switching cloud accounts. Syncing may terminate, but your app should go forth as though nothing happened.

And when your app is ready to reconnect to the cloud, Ensembles automatically migrates data to the cloud, so again, you are not required to play musical chairs with store files, and artificial migrations between stores, like you often do with other packages.

## Peer-to-Peer

Technically, Ensembles is not a peer-to-peer system, because servers are often used to store data. However, the *intelligent* work is all handled by the client framework, and the servers are really only used as a data transfer network, so Ensembles is closer to peer-to-peer than a standard client-server architecture. This is made even more apparent by the fact that it can also perform a traditional peer-to-peer sync, with no server involvement at all.

## Backend agnostic

The framework should work with any system capable of syncing up blobs of data (*ie* files) located at paths. Examples include, but are not limited to, iCloud, [Dropbox](https://www.dropbox.com)<sup>10</sup>, [S3](https://aws.amazon.com/s3/)<sup>11</sup>, [OmniPresence](https://www.omnigroup.com/omnipresence/)<sup>12</sup>, [WebDav](https://en.wikipedia.org/wiki/WebDAV)<sup>13</sup>, [FTP](https://en.wikipedia.org/wiki/File_Transfer_Protocol)<sup>14</sup>, Wi-Fi, and Bluetooth.

## Future Proof

Because with Ensembles you have access to the source code, and it is not married to any particular backend technology or service, it is as close to future proof as you can expect of any framework. If you integrate Ensembles, you have control over the source code, and the backend. If your backend ceases to exist, there are plenty more to choose from, and you can even evolve as new services come online.

## Files in the cloud are immutable

Ensembles ensures that when it adds a file to the cloud, it never moves it, or changes the file contents. Many of the issues that arise when using services like iCloud come from mutating a file on multiple devices. Making files immutable makes it much less likely that problems will be encountered.

---

<sup>10</sup>[http://dropbox.com](https://www.dropbox.com)

<sup>11</sup>[http://aws.amazon.com/s3/](https://aws.amazon.com/s3/)

<sup>12</sup>[http://www.omnigroup.com/omnipresence/](https://www.omnigroup.com/omnipresence/)

<sup>13</sup>[http://en.wikipedia.org/wiki/WebDAV](https://en.wikipedia.org/wiki/WebDAV)

<sup>14</sup>[http://en.wikipedia.org/wiki/File\\_Transfer\\_Protocol](https://en.wikipedia.org/wiki/File_Transfer_Protocol)

## Real time testing

Real time testing is essential for development, and for running automated tests. To achieve this goal, Ensembles supports backends that allow near instant transfer, such as the local file system and the pasteboard.

One of the great difficulties with testing other sync frameworks is that testing your app is mind-numbingly frustrating. You make a change on one device, and wait for it to propagate to another device where you can observe the effect of your change. This process can be minutes long.

## Eventual consistency of data across client devices

Having data be consistent across devices sounds like an obvious goal of a sync algorithm, but it is actually more difficult than it seems. We are dealing with a decentralized, peer-to-peer synchronization model, where no device can assume it has the complete global state at any point in time.

When merging a change set from a different device, it is often necessary to reconsider changes from a change set that has already been merged, in order to guarantee eventual consistency. Files can also arrive out of order, so an instruction to delete an object could appear before the object has even been created!

In order to ensure all devices apply and compare changes in the same order, whenever a new set of changes is first stored, the known revisions of all other devices are included. This forms a [vector clock](#)<sup>15</sup>, which allows Ensembles to establish exactly which change sets occurred concurrently, and provide a global ordering.

A centralised, server-based synchronization model, by contrast, such as the newly introduced [Dropbox Datastore](#)<sup>16</sup>, can make simplifying assumptions<sup>(1<sup>17</sup>2<sup>18</sup>)</sup> about what data needs to be included in merge operations.

## Conflict resolution handled in the spirit of Core Data

*Conflicts* arise when changes are made on two different devices without an intervening sync operation. Conflict resolution is tricky. You can flag every conflict, and require the host app to resolve each one. Or you can try to handle conflicts automatically, offering no insight or influence over what is happening. Ensembles adopts a completely new model of conflict resolution, tailored very much to the design ethos of Core Data.

It is common practice for Core Data developers to use a `NSManagedObjectContext` as a scratch pad for temporary data edits. Data can be manipulated, and is only required to be in a valid state when it is saved to the store. If a validation problem arises, it is possible to fix it, and retry.

This is a common pattern in Core Data apps, where a temporary context gets setup just for the purpose of making changes that may or may not end up committed to the store. Ensembles adopts this same approach for merging changes from other devices.

A temporary background `NSManagedObjectContext` is created which shares the same SQLite store as the main context. Change sets are merged into this context, in order, with no regard to validity of the object graph. When it is time to commit the changes, a delegate method is invoked to give the host app an opportunity to apply

---

<sup>15</sup>[http://en.wikipedia.org/wiki/Vector\\_clock](http://en.wikipedia.org/wiki/Vector_clock)

<sup>16</sup><https://www.dropbox.com/developers/datastore>

<sup>17</sup><https://www.dropbox.com/developers/blog/48/how-the-datastore-api-handles-conflicts-part-1-basics-of-offline-conflict-handling>

<sup>18</sup><https://www.dropbox.com/developers/blog/56/how-the-dropbox-datastore-api-handles-conflicts-part-two-resolving-collisions>

any *repairs* that it wants to make before the data is sent to disk. A second delegate method is called if the background save fails, offering another opportunity to make repairs. Any repairs made by the application code are *captured* and added as a change set.<sup>19</sup>

Many data models never require repairs, even after conflicts arise. For others, such as those with strict validation rules, repairs may be needed, but even then, they are usually localized to specific parts of the data model.

## Persistent store never in an invalid state

Ensembles allows you to stipulate the global identities of objects via a delegate callback. This has broad implications. It means that Ensembles can take on the responsibility of automatically adding data to the cloud when a store is first synchronized. It also means that data is automatically de-duplicated when logically identical objects are added on two different devices. This, and the repair mechanism discussed above, means that the persistent store should never be in an invalid state, and certainly not by design.

Other frameworks, such as Core Data's built in iCloud support, use a different approach. You have to de-duplicate objects yourself — after the changes have already been committed to the persistent store. Ensembles only commits to the persistent store once changes are known to be valid, and there are no duplicates.

## Large data blobs handled efficiently

Many apps these days store image, audio, and video files as binary NSData attributes in Core Data. Ensembles handles this data efficiently on low memory devices, and without duplication in the cloud. Large binary objects are stored in external files by the framework.

## Small cloud footprint

In order to sync, Ensembles must store files in the cloud. The size of this data is on the same order as the size of the store that is being synchronized, but every effort has been made to keep the footprint small. There is no duplication of data, and files are compact.

Ensembles also uses a technique called *baselining* to clean up old, redundant data. Baselining involves compacting cloud files into a single, new baseline. The new baseline contains only the insertion transactions needed to create a clone of the database.

Ensembles does this automatically when it detects that there is a significant reduction in data to be had. Ensembles is smart about how the baseline is stored, keeping data to a minimum, and avoiding making wholesale copies of the SQLite database.

## Graceful handling of model versions

Ensembles records the model version used to create each change set, so when merging changes from other devices, it can determine whether it can process the changes. If not, it gives an error and waits for the user to update the app. In the meantime, it continues to record any changes made by the user, and, after updating, sync continues with no data lost.

---

<sup>19</sup>This whole process is analogous to how a developer uses a DVCS like Git. Typically, you pull new versions from a server, and merge them with your local changes. If conflicts arise, you repair them, and commit these new changes, before pushing all local changes back to the server.

## Objective-C and Swift

Ensembles is written in Objective-C, but works fine with the new Swift language. Whether you are developing in Objective-C, Swift, or both, you can use Ensembles to add sync to your Core Data app.

## How Does it Work?

You should not need to know in detail how Ensembles synchronizes data stores in order to use it, but it certainly helps to have an idea of what it is doing. This understanding makes it easier to integrate the framework, and understand issues if any arise.

This section will give you a high level picture of how the framework operates, without going into any of the code level details.

## How it doesn't work

You may wonder why a sync framework is needed for Core Data at all. Couldn't we just upload the data store to the cloud whenever it changes, and download it on other devices?

There are a few problems with this that make it impractical. First, uploading the whole data store whenever a save occurs would result in excessive data transfer. This could be a big issue, particularly on mobile devices.

Second, it would be very difficult to merge the data if changes were made on two devices at roughly the same time. A new store may appear in the cloud from another device while the app is already running. You would have to find a way to load the new store, merge it with the existing store, and replace the local store all while the app is launched.

For these reasons, Ensembles and other frameworks like it take a different approach to sync.

## Transaction Logs

Ensembles breaks down each persistent store into a series of transactions. The transactions represent all of the insertions, updates, and deletions that go into building the store's database.

An insertion or update transaction is like an `NSDictionary`. It contains key-value pairs of an object's property names and values. For example, for a `Car` class, we may represent an object insertion like this JSON code

```
1 {  
2     "type" : "insert",  
3     "id" : "CAR12345",  
4     "manufacturer" : "Mazda",  
5     "model" : "626",  
6     "year" : 2003,  
7     "owner" : "OWNER54355"  
8 }
```

As you can see, the attributes of the class are stored as numerical or string values. Relationships, like the car's owner, are stored as identifiers that can be used to retrieve the related objects.

An update would be very similar to an insertion, but would only include the properties that changed.

```
1 {  
2   "type" : "update",  
3   "id" : "CAR12345",  
4   "owner" : "OWNER23368"  
5 }
```

In this example, the owner of the vehicle has been changed.

When an object is deleted, only the identifier of the object is needed, since the properties are irrelevant.

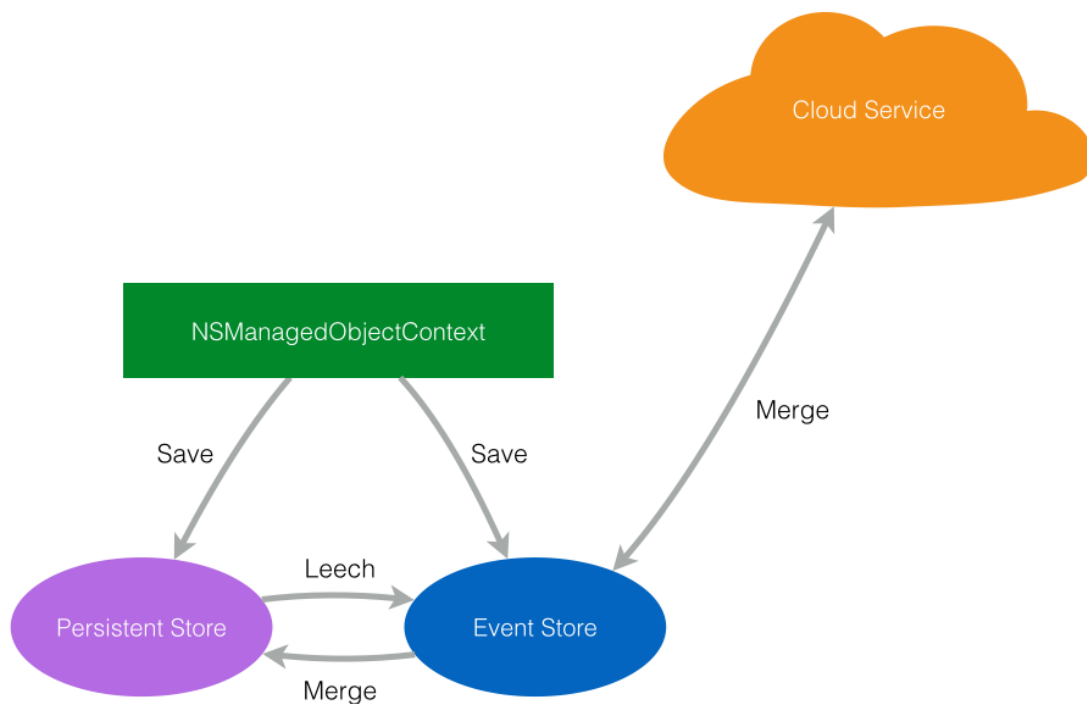
```
1 {  
2   "type" : "delete",  
3   "id" : "CAR12345"  
4 }
```

There are typically many of these changes made in a single save to the data store. All of these transactions get bundled up in files known as *transaction logs*. A timestamp is applied to each set of changes, so that they can be ordered later, and the files are what get uploaded to the cloud.

In other words, Ensembles records the changes to the data store — the so-called *deltas* — rather than copying the whole data store itself. By only transmitting the changes, the quantity of data transferred is considerably less, and it is also easier to combine changes from different devices: they just get *played back* in the same order as their timestamps dictate.

## The Big Picture

Now that the basic building blocks of the system are known (*i.e.* the transaction logs), we can consider the architecture of Ensembles, and flow of data through the system.



Architecture and data flow in Ensembles.

Ensembles extends the existing `NSManagedObjectContext` and `NSPersistentStore` of an app with a cloud service that is used to transport the transaction logs, and an *event store*. The event store is a local cache of the transaction logs; it is more optimal for working with the transactions than the individual files.<sup>20</sup>

Data flows between the various components as shown by the arrows in the figure. When the persistent store is first registered — a process known as *leeching* — its data is converted into insertion changes in the event store.

During a save of the main context, data is transferred to the persistent store as usual, but Ensembles also observes the save, and converts the changes into transactions in the event store.

When requested by the application code to merge the changes from other devices into the persistent store, Ensembles first retrieves new transaction logs from the cloud service and adds those to the event store. It then plays back the transactions, inserting, updating, and deleting entries in the persistent store as required. Merging also uploads locally created transactions to the cloud service so that other devices can merge them.

<sup>20</sup> A set of transactions arising from some modification of the persistent store, such as a save operation, is called an *event*. This explains the naming of the *event store*.



## The Event Store

As stated earlier, the event store is a local cache of transaction logs. It contains metadata used by the framework, and a Core Data SQLite store for the transactions. By importing the transactions into a single SQLite store, rather than accessing them directly in disparate transaction log files, merging can be made much more efficient.<sup>21</sup>

## Cloud File Systems

Ensembles is backend agnostic, so it can be made to work with many different cloud services. Each backend is represented by a class that conforms to the protocol `CDECLOUDFileSystem`. Backends are required to store data for file paths, much like a file system, hence the term *cloud file system*.

Although cloud file systems behave like simple file systems, there is no requirement that they actually be implemented as such; for example, you could create a cloud file system that stores data in a key-value store, with the paths used as keys.

## Leeching

Before a persistent store can be synced with other stores, the ensemble must *leech* it. *Leeching* is the process of registering the store with the other peers in the ensemble, and uploading the initial content of the local persistent store.

A store typically only needs to be leech once — it is persisted across app launches. You can explicitly *deleech* a store, if it should no longer be synced with its peers.

A spontaneous (forced) deleech can also occur in circumstances where the persistent store is permanently prevented from syncing, such as when the user logs in to a different account. Depending on the events that led to the deleech, you may be able to leech immediately to start syncing again.

## Merging

*Merging* is what most people would think of as syncing. It involves retrieving new data from other devices, merging it with local changes, applying those changes to the local store, and uploading any new transactions from the local device.

## Baselining

*Baselining* is the process of cleaning up old data. It only needs to run occasionally. When the framework detects that there is considerable data redundancy in the transaction logs, it compacts them into a new *baseline* file, which only includes the minimum number of (insertion) changes needed to rebuild the store.

---

<sup>21</sup>Having a separate event store can cause Ensembles to use more data storage than strictly necessary, but the performance benefits generally outweigh the storage concerns.

## Ensembles *versus* The Rest

There are a number of other sync solutions available for Core Data developers. This section compares some of the more commonly-used with Ensembles.

### Core Data's Native iCloud Support

The most obvious competitor to Ensembles is Apple's own built-in support for iCloud in Core Data. When this was first announced in 2011, many developers — including yours truly — were excited to try it. Unfortunately, the framework was buggy and poorly documented for the first two years, and this led many developers to give up in frustration.

iOS 7 and OS X 10.9 brought much needed improvements, and the framework seems much more stable than it was. However, even when Apple's solution works well, there are many issues which Apple — by nature of its size and agenda — will never be able to address. I'll enumerate some of them here.

The biggest problem with adopting the built-in sync is that it is closely coupled to iCloud. If you don't want to use iCloud, want to support multiple different backends, or are unable to use iCloud because your app is not in the App Store, you are out of luck. Apple don't support extensions, and are unlikely to do so in future.

Because Core Data only supports iCloud as a backend, you can't have multi-user sync. Ensembles has no restrictions in this regard; if you have a backend that manages multiple users, you can have them share and sync stores.

Core Data is a proprietary framework, and you don't have access to the source code. When an issue arises, it can be very difficult to determine what is going wrong. If you discover a bug, it is very likely to be a year before Apple can address the issue in the next major system upgrade. In an project like Ensembles, bug fixes can be issued in a matter of hours.

The debugging facilities for iCloud in Xcode don't allow for local testing, so you always have to push data via iCloud when developing. This can slow things down considerably, and make setting up automated tests much more difficult. Ensembles supports syncing via the local file system, and includes test suites built upon this feature.

The design of the Core Data-iCloud API makes it troublesome to adopt. There is an overly tight coupling between the Core Data stack and the sync functionality, so you will often find persistent stores being added and removed by the framework as your app runs.

Core Data will not migrate your data to the cloud, or do any merging of data. Instead, you have to deduplicate data after it has already been incorporated in the persistent store, using fetch requests to locate corresponding objects. Ensembles gives you the ability to provide global identifiers for objects, which are used to automatically migrate and merge data.

Lastly, Core Data provides no hooks to resolve conflicts. If a conflict occurs, the behavior is undocumented. Will the data be deleted? Will the transaction be rolled back? It is not clear, and the developer has no control over the outcome. Ensembles includes delegate methods where you can 'repair' objects invalidated by conflicting changes.

## CloudKit

Apple introduced a new option for cloud storage at WWDC 2014: CloudKit. CloudKit is a promising technology, supporting not only data transfer between a user's devices, but also sharing of data between users. It opens up a whole range of possibilities for app developers.

Although, in theory, you can sync Core Data stores with CloudKit, there is no built in support, and you will need to implement a lot of the sync algorithms yourself. In effect, you will be reimplementing large parts of the Ensembles framework, including local change tracking, mapping of cloud records to Core Data managed objects, and conflict resolution. Where it may only take 100 lines of code to get your app syncing with Ensembles, it may take several thousand lines to achieve the same with CloudKit.

## Tim Isted Core Data Sync (TICDS)

Before Ensembles, and even before iCloud support was added to Core Data, Tim Isted developed an open source framework known as [TICDS](#)<sup>22</sup>. This framework blazed a trail for later projects like Ensembles, and is still used in quite a few apps today.

In technical terms, TICDS is very similar to Ensembles, supporting Core Data sync via file transfer backends like Dropbox and iCloud. Like Ensembles, it can also be extended to support any backend capable of transferring file data.

Unfortunately for the TICDS project, Tim has moved on to work at Apple, and is unable to continue to develop it. The project is entering a state of gradual deterioration. The design of certain parts of the TICDS framework have also been improved upon in Ensembles, aided by the lessons learnt from it.

## Wasabi Sync

[Wasabi Sync](#)<sup>23</sup> is a hosted service for Core Data apps. It is well regarded, and used in apps like Bare Bone's [Yojimbo](#)<sup>24</sup>. Unfortunately, you can no longer purchase a new developer account for Wasabi Sync.

## Simperium

[Simperium](#)<sup>25</sup> is a service that arose out of the [Simplenote](#)<sup>26</sup> app. It has a Core Data interface, though it does not seem to be well maintained.

Simperium is a cross platform cloud solution. Being a hosted solution, you have to pay for online storage.

## Dropbox Datastore API

Recently, Dropbox introduced its [Datastore API](#)<sup>27</sup>. By adopting their SDK for your model layer, you can have your data mirrored across devices.

The Datastore API doesn't support Core Data directly, but the open source [ParcelKit](#)<sup>28</sup> project offers a bridge.

---

<sup>22</sup><https://github.com/nothirst/TICoreDataSync>

<sup>23</sup><http://www.wasabisync.com>

<sup>24</sup><http://www.barebones.com/products/yojimbo/>

<sup>25</sup><http://simperium.com>

<sup>26</sup><http://simplenote.com>

<sup>27</sup><https://www.dropbox.com/developers/datastore>

<sup>28</sup><https://github.com/overcommitted/ParcelKit>

An advantage of the Datastore API over other providers is that the developer doesn't have to pay for storage. Dropbox customers have already paid for storage, and your app can leverage it free of charge.

## Parse

[Parse](https://parse.com)<sup>29</sup>, which is now owned by Facebook, gives you an SDK similar to Dropbox's Datastore API. A major difference with Dropbox is that you have to pay hosting costs.

Parse does not support Core Data directly, but there are [open source projects](https://github.com/itsniper/FTASync)<sup>30</sup> to bridge the gap.

As with any service that requires you to adopt a new API, you cannot easily migrate away from Parse without rewriting your model code.

## BaaSBox

[BaasBox](http://www.baasbox.com)<sup>31</sup> is an open source variant of Parse. You can run your own server, or pay for hosting.

BaaSBox is cross platform, but doesn't have any Core Data integration. Like Parse and the Datastore API, you have to adopt a new API for your model objects.

## Realm.io

[Realm](http://realm.io)<sup>32</sup> is a new cross platform data store, with support for sync. It is off to a very promising start, but makes no use of Core Data — it is a replacement for Core Data. Moving to Realm means moving away from Core Data; if you want to stick with Core Data, Ensembles offers a robust way to sync your data across devices.

---

<sup>29</sup><https://parse.com>

<sup>30</sup><https://github.com/itsniper/FTASync>

<sup>31</sup><http://www.baasbox.com>

<sup>32</sup><http://realm.io>

# Installing Ensembles

Before integrating Ensembles into your Xcode project, you need to download it. There are various forms the framework comes in, and different ways to get it. This chapter describes how you can download and install Ensembles.

## Downloading Ensembles

### Migrating from Ensembles 1.x to Ensembles 2

Ensembles 2 is a drop-in replacement for Ensembles 1.x — the API has been extended, but is fully compatible — so it's easy to start development with Ensembles 1.x, and move to Ensembles 2 before shipping.

If you already have a shipping app using Ensembles 1.x, you need to be aware that although Ensembles 2 has an API that is compatible with Ensembles 1.x, it is not binary compatible. In particular, the cloud file storage format is changed, so you should not mix Ensembles 1.x and Ensembles 2 cloud data. The easiest way to migrate is simply to deleech, remove the cloud data, and create a new Ensemble under a different name (e.g. MainStore.v2).

### Support Package Downloads

When you purchase a support package for Ensembles, it includes source code for the framework, as well as easy-to-install binaries. Just unzip the package, and navigate to the appropriate directory.

### Cloning Ensembles with Git

If you like to stay up-to-date with the latest changes to the source code, you can use Git and the GitHub web site to install and manage Ensembles.

To clone the Ensembles 1.x repository to your local drive, use the command

```
1 git clone https://github.com/drewmccormack/ensembles.git
```

If you are using Ensembles 2, you will first need to request access to the private GitHub repository. Once you have access, you can clone using the command

```
1 git clone https://github.com/mentalfaculty/ensembles-next.git ensembles
```

Ensembles makes use of Git submodules. To retrieve these, change to the `ensembles` root directory in Terminal

```
1 cd ensembles
```

and issue this command

```
1 git submodule update --init
```

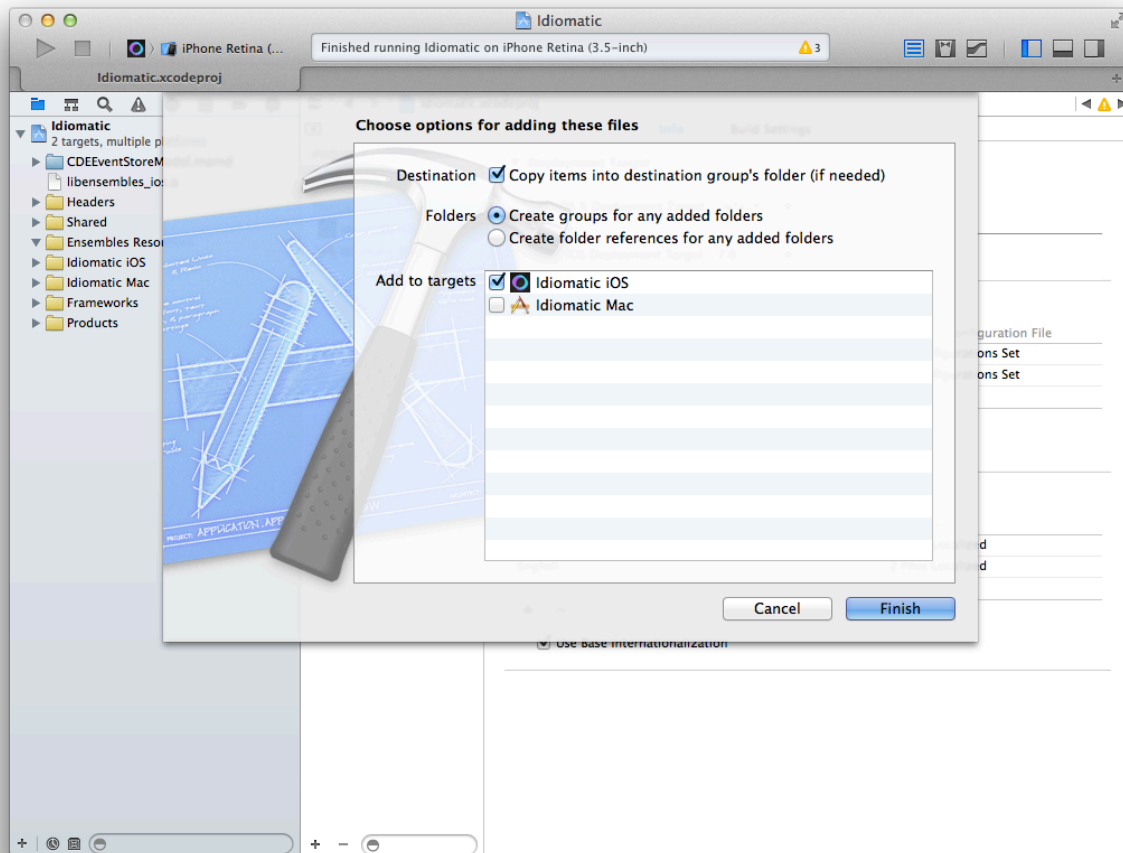
If you think you may want to contribute source code back to the Ensembles project at some point, it may be worth forking the project on GitHub, and cloning your fork instead of the main repository. This will allow you to issue pull requests on GitHub.

## Integrating Ensembles into an Xcode Project

### Installing the Binaries

If you purchased a support package for Ensembles, it should include ready-made binaries for the framework, which make installing as easy as drag-and-drop. If you have the binaries, here is how you can integrate them into your Xcode project.

1. Drag the framework bundle *Ensembles.framework* from the iOS or OS X folder into your Xcode project source list.
2. In the sheet that appears, make sure you check the *Copy items into destination group's folder* option, make sure your app target is checked, and then click *Finish*.



#### Adding the library.

3. For an iOS app, drag the resources bundle *Ensembles.bundle* into your Xcode source list. Make sure you check the *Copy items into destination group's folder* option, and check the box for the app's target.
4. For a Mac app, create a new build phase to copy frameworks into your app bundle (if you don't already have one). To do this...
  - A. Select the project root in the source list, then select your app's target.
  - B. Open the *Build Phases* tab.
  - C. Click the + button at the top of the list.
  - D. Choose *New Copy Files Build Phase* from the popup menu.
  - E. Disclose the contents of the new *Copy Files* phase, and choose *Frameworks* from the *Destination* popup button.
  - F. Click the + button at the bottom of the *Copy Files* phase section, choose *Ensembles.framework*, and click *Add*.
5. For an iOS project, select the *Build Settings* tab of the app target. Locate the *Other Linker Flags* setting, and add the flag *-ObjC*.
6. For a Mac app, locate the *Runpath Search Path* build setting, and add *@loader\_path/../../Frameworks*.
7. If you need to install other backends, such as Dropbox, drag the relevant files from the *Extra Backends* folder into Xcode.

## Using CocoaPods for Ensembles 1.x

[CocoaPods](http://cocoapods.org)<sup>33</sup> is a package management tool that has become popular amongst iOS developers. Ensembles includes support for CocoaPods, making installation a breeze. Here is how you add Ensembles 1.x to your App's Xcode Project with CocoaPods.

Add the following to your project's Podfile

```
1 platform :ios, '7.0'
2 pod "Ensembles", "~> 1.0"
```

This will install Ensembles with the iCloud backend.

To use other cloud services, such as Dropbox, add the relevant subspec to the Podfile. For example, to include Dropbox, include

```
1 pod "Ensembles/Dropbox", "~> 1.0"
```

CocoaPods subspecs supported in Ensembles 1.x.

| Identifier          | Supported Backends                                                   |
|---------------------|----------------------------------------------------------------------|
| Ensembles/Core      | Local file system and iCloud.                                        |
| Ensembles/Dropbox   | All in Core, and Dropbox Core API.                                   |
| Ensembles/Multipeer | All in Core, and Multipeer Connectivity.                             |
| Ensembles/Node      | All in Core, and Node.js server supplied with some support packages. |

## Using CocoaPods for Ensembles 2

To use CocoaPods for Ensembles 2, you first need to request access to the private GitHub repository.

Once you have access, you can add the private repository of The Mental Faculty to your Cocoapods installation.

```
1 pod repo add mentalfaculty https://github.com/mentalfaculty/Specs.git
```

Once you have done that, you can include Ensembles 2 in your Podfile.

---

<sup>33</sup><http://cocoapods.org>



```

1 source 'https://github.com/mentalfaculty/Specs.git'
2 source 'https://github.com/CocoaPods/Specs.git'
3
4 platform :ios, '7.0'
5 pod "Ensembles", "~> 2.0"

```

Including optional backends is the same as for Ensembles 1.x, except that there are more to choose from.

CocoaPods subspecs supported in Ensembles 2.

| Identifier          | Supported Backends                                                   |
|---------------------|----------------------------------------------------------------------|
| Ensembles/Core      | Local file system and iCloud.                                        |
| Ensembles/CloudKit  | All in Core, and CloudKit.                                           |
| Ensembles/Dropbox   | All in Core, and Dropbox Core API.                                   |
| Ensembles/WebDAV    | All in Core, and WebDAV.                                             |
| Ensembles/Multipeer | All in Core, and Multipeer Connectivity.                             |
| Ensembles/Zip       | All in Core, and Zip compression.                                    |
| Ensembles/Encrypt   | All in Core, and encryption of cloud data.                           |
| Ensembles/Node      | All in Core, and Node.js server supplied with some support packages. |

## Manually Installing in Xcode (iOS)

If you like to control all aspects of installing the source code yourself, follow the following procedure to add Ensembles to your iOS App's Xcode Project.

1. In Finder, drag the `Ensembles iOS.xcodeproj` project from the `Framework` directory into your Xcode project.
2. Select your App's project root in the source list on the left, and then select the App's target.
3. In the General tab, click the + button in the *Linked Frameworks and Libraries* section.
4. Choose the `libensembles.a` library and add it.
5. Select the *Build Settings* tab. Locate the *Other Linker Flags* setting, and add the flag `-ObjC`.
6. Select the *Build Phases* tab. Open *Target Dependencies*, and click the + button.
7. Locate the `Ensembles Resources iOS` product, and add that as a dependency.
8. Open the `Ensembles iOS.xcodeproj` project in the source list, and open the Products group.
9. Drag the `Ensembles.bundle` product into the *Copy Bundle Resources* build phase of your app.
10. Add the following import in your precompiled header file, or in any files using Ensembles.

```
1 #import <Ensembles/Ensembles.h>
```

By default, Ensembles only includes support for iCloud. To use other cloud services, such as Dropbox, you need to locate the source files and frameworks relevant to the service you want to support. You can find frameworks in the *Vendor* folder, and source files in *Framework/Extensions*. More information on installing other backends is available in the [chapter on standard backends](#).

By way of example, if you want to support Dropbox, you need to add the DropboxSDK Xcode project as a dependency, link to the appropriate product library, and include the files *CDEDropboxCloudFileSystem.h* and *CDEDropboxCloudFileSystem.m* in your project.

## Manually Installing in Xcode (OS X)

Follow the following procedure to add Ensembles to your OS X App's Xcode Project.

1. In Finder, drag the `Ensembles Mac.xcodeproj` project from the `Framework` directory into your Xcode project.
2. Select your App's project root in the source list on the left, and then select the App's target.
3. In the General tab, click the + button in the *Linked Frameworks and Libraries* section.
4. Choose `Ensembles.framework` and add it.
5. Create a new build phase to copy frameworks into your app bundle (if you don't already have one). To do this...
  - A. Select the project root in the source list, then select your app's target.
  - B. Open the *Build Phases* tab.
  - C. Click the + button at the top of the list.
  - D. Choose *New Copy Files Build Phase* from the popup menu.
  - E. Disclose the contents of the new *Copy Files* phase, and choose *Frameworks* from the *Destination* popup button.
  - F. Click the + button at the bottom of the *Copy Files* phase section, choose *Ensembles.framework*, and click *Add*.
6. Locate the *Runpath Search Path* build setting, and add `@loader_path/../Frameworks`.
7. Add the following import in your precompiled header file, or in any files using Ensembles.

```
1  #import <Ensembles/Ensembles.h>
```

To use cloud services other than iCloud, locate the frameworks in the *Vendor* folder, and source files in *Framework/Extensions*.

## Code Signing on OS X

When you come to distribute your Mac OS X app, you will likely want to code sign it. Embedded frameworks, like Ensembles, also have to be code signed.

The latest versions of Xcode allow you to check a box in the Copy Build Phase to sign the framework when installing. If you don't have this option, or want to control code signing yourself, you can instead add a new *Run Script* build phase to your target. Make sure it is the last build phase, and add the following script.

```
1 LOCATION="${BUILT_PRODUCTS_DIR}/${FRAMEWORKS_FOLDER_PATH}"
2 RESOURCES_LOCATION="${BUILT_PRODUCTS_DIR}/${UNLOCALIZED_RESOURCES_FOLDER_PATH}"
3 IDENTITY="<Name of your code signing certificate here>"
4
5 codesign --verbose --force --sign "$IDENTITY" "$LOCATION/Ensembles.framework/Versions/A"
6 codesign --verbose --force --sign "$IDENTITY" "$RESOURCES_LOCATION/Ensembles.bundle"
```

You will need to fill in the name of your code signing certificate for the `IDENTITY` variable. In practice, it doesn't really matter what identity you use to sign the framework. It seems that Mac OS X requires that the embedded frameworks be signed, but doesn't care by whom.

## Idiomatic App

Idiomatic is a relatively simple example app which incorporates Ensembles and works with a selection of different backends to sync across devices. The app allows you to record your ideas, takes photos, and add tags for grouping. The Core Data model of the app includes three entities, with many-to-many and many-to-one relationships.

The Idiomatic project is a good way to get acquainted with Ensembles, and how it is integrated in a Core Data app. Idiomatic can be run in the iPhone Simulator, or on a device, but in order to test it, you need to follow a few preparatory steps.

## Supporting iCloud Sync

To support iCloud sync in Idiomatic, follow these steps.

1. Select the Idiomatic Project in the source list of the Xcode project, and then select the Idiomatic target.
2. Select the *Capabilities* section, turn on the iCloud switch.
3. Build and install on devices and simulators that are logged into the same iCloud account.

Add notes, and tag them as desired. The app will sync when it becomes active, but you can force a sync by tapping the button under the Groups table.

## Supporting Dropbox

Dropbox should work via The Mental Faculty account, but if you want to use your own developer account, you need to do the following:

1. Sign up for an account at the [Dropbox Developer Site](http://developer.dropbox.com)<sup>34</sup>.
2. In the App Console, click the *Create app* button.
3. Choose the Dropbox API app type.
4. Choose to store *Files and Datastores*

---

<sup>34</sup><http://developer.dropbox.com>

5. Choose *Yes — My app only needs access to files it creates*
6. Name the app (eg *Idiomatic*)
7. Click on *Create app*
8. At the top of the *IDMSyncManager.m* file, locate this code, and replace the values with the strings you just created on the Dropbox site.

```

1      NSString * const IDMDropboxAppKey = @"xxxxxxxxxxxxxxxx";
2      NSString * const IDMDropboxAppSecret = @"xxxxxxxxxxxxxxxx";

```

1. Select the Idiomatic project in Xcode, and then the Idiomatic iOS target.
2. Select the Info tab.
3. Open the URL Types section, and change the URL Schemes entry to

```

1      db-<Your Dropbox App Key>

```

## Deploying Ensembles Server

Some support packages include access to a custom server written with [Node.js](http://nodejs.org)<sup>35</sup> and backed by Amazon's S3 online storage. You can deploy this web service on [Heroku](http://heroku.com)<sup>36</sup> if you want a custom sync service.

### What is Ensembles Server?

Ensembles Server is a basic Node.js web app that utilizes Amazon's [S3](http://aws.amazon.com)<sup>37</sup> for file storage, and a Postgres database for user management. It can be used with the `CDNodeCloudFileSystem` class in the Ensembles framework on client devices.

The server offers HTTP Basic Authentication, and an API that supports user actions such as signing up, logging in, changing password, and resetting a password (emails new password). The Idiomatic iOS app gives an example of how an interface for these user actions can be setup, and integrated with the `CDNodeCloudFileSystem` class.

The server also has APIs used by `CDNodeCloudFileSystem` to upload and download files to/from S3. The Node.js server doesn't handle file transfers directly; rather, it returns time-limited signed URLs, and the `CDNodeCloudFileSystem` uses these URLs to exchange data directly with S3.

---

<sup>35</sup><http://nodejs.org>

<sup>36</sup><http://heroku.com>

<sup>37</sup><http://aws.amazon.com>

## Prerequisites

You will need:

- An [Amazon AWS account](#)<sup>38</sup>, and a bucket in S3 called ‘ensembles-server’
- An account at [Heroku.com](#)<sup>39</sup>

You can also host your Node.js server on cloud services other than Heroku, but the instructions may differ some.

## Installing a Development Mac

- Download and install the [Heroku Toolbelt](#)<sup>40</sup>
- Download and install Node.js from [nodejs.org](#)<sup>41</sup>
- Download and install [Postgres.app](#)<sup>42</sup>

## Running Ensembles Server on Development Mac

Setup your environment by editing your `.profile` or `.bash_profile` configuration file. Include your AWS credentials, and the path to the Postgres database tools.

```
1 export AWS_ACCESS_KEY_ID=<Your AWS Key>
2 export AWS_SECRET_ACCESS_KEY=<Your AWS Secret>
3 export PATH=$PATH:/Applications/Postgres.app/Contents/Versions/9.3/bin
4 export NODE_ENV=development
```

At this point, you should ‘source’ the configuration file, or exit the shell and launch a new one.

Ensembles Server stores its data in a bucket on Amazon’s S3. Create a bucket for the server using the AWS web interface, and then fill in the bucket name for `process.env.BUCKET` in the `server.js` file. (Note there are two places this is set.)

Now to create a database. Launch Postgres.app, and then issue this command in Terminal.

```
1 createdb ensembles-server
```

Install the Node.js packages by changing to the root directory of the Ensembles Server project, and issuing this command

---

<sup>38</sup><http://aws.amazon.com>

<sup>39</sup><http://heroku.com>

<sup>40</sup><https://toolbelt.heroku.com>

<sup>41</sup><http://nodejs.org>

<sup>42</sup><http://postgresapp.com>

```
1 npm install
```

Now you can run the server on your Mac. Just issue this command

```
1 foreman start
```

This will launch the Node.js server, and listen on port 5000 of the Mac.

You can dispatch HTTP requests using CLI tools like `curl`, and Mac apps like Paw and Rest Client. Use URLs beginning `http://localhost:5000`.

If you would like to test Idiomatic with your development server, locate the method `makeCloudFileSystem` in the `IDMSyncManager` class, and change the base URL used to initialize the `CDNodeCloudFileSystem` to `http://localhost:5000`

In practice, you are probably best just moving straight on to using Idiomatic with Heroku, which is described next.

## Setting Up Heroku

If you haven't already created an S3 bucket for your app, create one now using the AWS web interface, and then fill in the bucket name for `process.env.BUCKET` in the `server.js` file.

Apps are installed on Heroku via Git, so you need to setup your Ensembles Server installation as a Git repository. Open Terminal and issue the following commands from the root directory of the Ensembles Server code base.

```
1 git init
2 git add .
3 git commit -m "First commit"
```

Login to Heroku on the command line.

```
1 heroku login
```

Accept if it asks if you want to generate an SSH key.

Create an app on Heroku with a unique name.

```
1 heroku apps:create <Name of App>
```

Include Heroku as a remote repository, so that you can push the Ensembles Server code to it. Use the name you chose above.

```
1 git remote add heroku git@heroku.com:<Name of App>.git
```

Add a PostgreSQL database.

```
1 heroku addons:add heroku-postgresql:dev
```

Add [SendGrid](https://sendgrid.com)<sup>43</sup>, which is used to email new passwords. (You may need to supply Heroku with a credit card in order to add this package, even if you are using a free plan.)

```
1 heroku addons:add sendgrid
```

Set the Amazon AWS credentials in the Heroku environment.

```
1 heroku config:set AWS_ACCESS_KEY_ID=<Your Access Key> AWS_SECRET_ACCESS_KEY=<Your Secret Key>
2
```

Configure Heroku as your production environment.

```
1 heroku config:add NODE_ENV=production
```

Push your code to Heroku with Git.

```
1 git push heroku master
```

You can check what dynos are running on Heroku using

```
1 heroku ps
```

To start a single dyno, run this command

```
1 heroku ps:scale web=1
```

To check for errors, issue this command

```
1 heroku logs
```

You should now be able to change the base URL used by the `CDNodeCloudFileSystem` in `Idiomatic`, or your own app, to use the URL of your Heroku app.

---

<sup>43</sup>[http://sendgrid.com](https://sendgrid.com)

# Quick Start Guide

You don't like to waste time. I get it. So what's keeping you?

## E.L.M.

The basic steps required to start syncing a persistent store are captured in the acronym E.L.M.: Ensemble, Leech, Merge. The next few sections describe what is involved.

### Ensemble

The most important class in the Ensembles framework is `CDEPersistentStoreEnsemble`. You create one instance of this class for each `NSPersistentStore` that you want to sync. This class monitors saves to your SQLite store, and merges in changes from other devices as they arrive.

You typically initialize a `CDEPersistentStoreEnsemble` at about the same point in your code that your Core Data stack is initialized. It is important that an ensemble — once it has been leeches — is initialized before your app performs any saves.

There is one other family of classes that you need to be familiar with. These are classes that conform to the `CDECloudFileSystem` protocol. Any class conforming to this protocol can serve as the file syncing backend of an ensemble, allowing data to be transferred between devices. You can use one of the existing classes (eg, `CDEICloudFileSystem`), or develop your own.

The initialization of an ensemble is typically only a few lines long.

```
1 // Setup Ensemble
2 cloudFileSystem = [[CDEICloudFileSystem alloc]
3     initWithUbiquityContainerIdentifier:@"P7BXV6PHLD.com.mentalfaculty.idiomatic"];
4 ensemble = [[CDEPersistentStoreEnsemble alloc] initWithEnsembleIdentifier:@"MainStore"
5     persistentStoreURL:storeURL
6     managedObjectModelURL:modelURL
7     cloudFileSystem:cloudFileSystem];
8 ensemble.delegate = self;
```

After the cloud file system is initialized, it is passed to the `CDEPersistentStoreEnsemble` initializer, together with the URL of a file containing the `NSManagedObjectModel`, and the file URL for the `NSPersistentStore`. An ensemble identifier is used to match stores across devices. It is important that this be the same for each store in the ensemble.



## Leech

Once a `CDEPersistentStoreEnsemble` has been initialized, it can be *leached*. This step typically only needs to take place once, to setup the ensemble and perform an initial import of data in the local persistent store. Once an ensemble has been leached, it remains leached even after a relaunch. The ensemble only gets *deleached* if you explicitly request it, or if a serious problem arises in the cloud file system, such as an account switch.

You can query an ensemble for whether it is already leached using the `isLeeched` property, and initiate the leeching process with `leechPersistentStoreWithCompletion:`. (Attempting to leech an ensemble that is already leached will cause an error.)

```
1  if (!ensemble.isLeeched) {
2      [ensemble leechPersistentStoreWithCompletion:^(NSError *error) {
3          if (error) NSLog(@"Could not leech to ensemble: %@", error);
4      }];
5  }
```

Because many tasks in Ensembles can involve networking or long operations, most methods are asynchronous and include a block callback which is called on completion of the task with an error parameter. If the error is `nil`, the task completed successfully. Methods should only be initiated on the main thread, and completion callbacks are sent to the main queue.

## Merge

With the ensemble leached, sync operations can be initiated using the `mergeWithCompletion:` method.

```
1  [ensemble mergeWithCompletion:^(NSError *error) {
2      if (error) NSLog(@"Error merging: %@", error);
3  }];
```

A merge involves retrieving new changes for other devices from the cloud file system, integrating them in a background `NSManagedObjectContext`, merging with new local changes, and saving the result to the `NSPersistentStore`.

When a merge occurs, it is important to merge the changes into your main `NSManagedObjectContext`. First, you need to make sure you have set the `mergePolicy` of your context (eg, `NSMergeByPropertyStoreTrumpMergePolicy`).

Then, you can do this in the `persistentStoreEnsemble:didSaveMergeChangesWithNotification:` delegate method (or the equivalent notification handler).

```

1 - (void)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble
2   didSaveMergeChangesWithNotification:(NSNotification *)notification
3 {
4     [managedObjectContext performBlock:^(
5         [managedObjectContext mergeChangesFromContextDidSaveNotification:notification];
6     )];
7 }

```

Note that this is invoked on a background thread. You need to make sure the `mergeChangesFromContextDidSaveNotification:` method is invoked on the thread corresponding to the main context. You should also merge the changes into any other contexts that depend in any way on the persistent store, such as a child context of the main context.

## Using Ensembles with Magical Record

[Magical Record<sup>44</sup>](#) is a popular open-source framework used by many Core Data developers. Integrating Ensembles into a project with Magical Record is almost the same as for ‘vanilla’ Core Data projects.

First, you need to alter how you load the Entity Model file. Usually, Magical Record will load your model for you, and will merge all model files together into a single `NSManagedObjectContext`. Because Ensembles has its own internal Core Data model, you will want to prevent Magical Record from merging models.

Instead, have it only load your application model, like this

```

1 [MagicalRecord setShouldAutoCreateManagedObjectContext:NO];
2 NSManagedObjectContext *model = [NSManagedObjectContext MR_newManagedObjectContextNamed:@"Recipe\
3 s.momd"];
4 [NSManagedObjectContext MR_setDefaultManagedObjectContext:model];
5 [MagicalRecord setupAutoMigratingCoreDataStack];

```

To get the location of your model file, which is needed to setup Ensembles, just use the `NSBundle` class to locate the `.momd` file corresponding to the Entity Model you created. For example, if your Entity Model file is called `Recipes.xcdatamodeld`, you would retrieve the file URL like this

```

1 NSURL *modelURL = [[NSBundle mainBundle] URLForResource:@"Recipes" withExtension:@"momd"];

```

In addition to the model file URL, you also need to know the location of your persistent store. Magical Record can give you the location. If you are using a custom name for your store, pass in the name that you passed to `setupCoreDataStackWithStoreNamed:`.

```

1 NSURL *storeURL = [NSPersistentStore MR_urlForStoreName:@"MyStore.sqlite"];

```

If you are using the default store name, use the `MR_defaultLocalStoreUrl` method instead.

---

<sup>44</sup><https://github.com/magicalpanda/MagicalRecord>

```
1  NSURL *storeURL = [NSPersistentStore MR_defaultLocalStoreUrl];
```

Once you have the file URL for the persistent store, and the file URL for the Entity Model, you can proceed to setup the ensemble. You should do this just after you setup your Magical Record stack, and before your app saves data.

```
1  // Setup Ensemble
2  cloudFileSystem = [[CDEICloudFileSystem alloc]
3      initWithUbiquityContainerIdentifier:@"XXXXXXXXXX.com.yourcompany.yourapp"];
4  ensemble = [[CDEPersistentStoreEnsemble alloc]
5      initWithEnsembleIdentifier:@"YourStoreIdentifier"
6      persistentStoreURL:storeURL
7      managedObjectModelURL:modelURL
8      cloudFileSystem:cloudFileSystem];
9  ensemble.delegate = self;
```

You initiate leeching and merging just as for other Core Data projects, as described earlier in the chapter.

When a merge completes, it commits changes from other devices to the persistent store. It is important to merge these changes into the `NSManagedObjectContext`s under the control of Magical Record. If you do not make them aware of these changes, the new data will not be accessible, and you will likely get an error when you next try to save.

To merge the data, implement the `persistentStoreEnsemble:didSaveMergeChangesWithNotification:` method, invoking the `mergeChangesFromContextDidSaveNotification:` method for each context.

```
1  - (void)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble
2      didSaveMergeChangesWithNotification:(NSNotification *)notification
3  {
4      NSManagedObjectContext *rootContext = [NSManagedObjectContext MR_rootSavingContext];
5      [rootContext performBlock:^(
6          [rootContext mergeChangesFromContextDidSaveNotification:notification];
7      )];
8
9      NSManagedObjectContext *mainContext = [NSManagedObjectContext MR_defaultContext];
10     [mainContext performBlock:^(
11         [mainContext mergeChangesFromContextDidSaveNotification:notification];
12     )];
13 }
```

The code above should work for a typical Magical Record Core Data stack. If you have a different setup, it is important that all contexts that depend in some way on the content of the persistent store merge the changes from the notification.

For a working example incorporating Magical Record and Ensembles, see the *Magical Record* project in the *Examples* folder of the project.

## Global Identifiers

The steps in E.L.M. described above are all of the steps you need to get a basic sync up and running. But it is advisable to add one more step to make the whole thing work seamlessly: you need to tell Ensembles the identity of your objects.

### Global Identity vs De-duplication

By default, Ensembles will create random identifiers for all of the objects it imports. This is also how Core Data-iCloud sync works. The disadvantage of this is that if the same object is inserted on two different devices, you end up with duplicates, and you will have to manually fetch to search for and remove the duplicates.

This involves a lot of code and hair pulling, so Ensembles allows you to give identities to your objects, so that it can match objects on one device with those on another. It then knows which ones correspond, and can update the appropriate object. No more duplicates.

### Providing Global Identifiers

To provide global identifiers for your objects, you need to implement a delegate method of the `CDEPersistentStoreEnsemble` class. Assuming you have added a `uniqueIdentifier` attribute to your managed objects, this method could be as simple as this:

```
1 - (NSArray *)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble
2     globalIdentifiersForManagedObjects:(NSArray *)objects
3 {
4     return [objects valueForKeyPath:@"uniqueIdentifier"];
5 }
```

The method is invoked on a background thread. Care should be taken to only access the objects passed on this thread.

Global identifiers need to be unique for any entity. You can have the same identifier used for different entities. For example, if you have an entity called *Cat*, and another called *Dog*, it would be OK to have a *Cat* object with the identifier 'snoopy', and a *Dog* also with the identifier 'snoopy'. But there should not be two *Dog* objects with the identifier 'snoopy'.

Global identifiers should also be immutable. A managed object should never change its global identifier once it has been assigned. For example, if you were to use the email address of a *User* entity as the global identifier, you would need to be sure that the user could never modify the address.

If you are adding Ensembles to an existing app, and you are performing a version migration in order to include global identifiers, it is important that these identifiers are initialized and saved *before* you perform the first leech. You may need to manually fetch your data after the migration, in order to initialize and save the global identifiers.

Finally, no two objects in a single save operation should share the same global identifier, even if one object has been deleted. Until the save operation completes, the Ensembles framework is unaware of the deletion. If you delete an object, you need to save the changes to the persistent store before inserting a new object with the same global identifier. See the chapter on [Saving](#) for more information.

## Storing Global Identifiers

It is up to you how you generate the global identifiers, and where you store them. A common choice is to add an extra attribute to entities in your data model, and set that to a uuid on insertion into the store.

Imagine you add an `NSString` attribute to your *Cat* entity, and give it the name *uniqueIdentifier*. You could initialize the attribute in `awakeFromInsert`.

```
1 - (void)awakeFromInsert
2 {
3     [super awakeFromInsert];
4     if (!self.uniqueIdentifier) self.uniqueIdentifier = [[NSUUID UUID] UUIDString];
5 }
```

We first check that the unique identifier has not yet been set. This is necessary because, unfortunately, it is possible for `awakeFromInsert` to be invoked twice when using parent-child relationships for `NSManagedObjectContext`.

If we create a `uniqueIdentifier` attribute or getter method for each of our `NSManagedObject` subclasses, we can just return the unique identifiers from the delegate method as shown [above](#).

## Choosing Global Identifiers

For many objects, it makes sense to use random global identifiers. The `NSUUID` class, or `NSProcessInfo` method `globallyUniqueString`, are useful for this purpose.

But for some entities, a random identifier is not appropriate. When objects come from a fixed set, you should be careful how you assign identifiers.

For example, sometimes an app will have an entity for which there should only ever be one instance. A singleton if you will. A common use case is to store metadata for the app in this entity. In this case, there clearly should never be more than one instance, so it makes sense to assign it a single identifier (eg “Metadata”), so that Ensembles will treat each instance as being one and the same.

There are other cases where you need to be careful about assigning particular identifiers. Take the case of an entity that represents a tag. The identity of a tag is tied to its string. There should only be one `Tag` object for the string “car”, so “car” is actually a good identifier to choose. If you use the tag’s string as its global identifier, Ensembles can establish that the `Tag` object on one device with the identifier “car” actually represents the same logical instance as a `Tag` object on another device with that same identifier. It will thus avoid duplication of the tag, even if there are multiple inserts on different devices.

Take one last example: a fixed set of colors. Maybe you are representing color by an entity, but you have a fixed number to choose from. You could have a `Color` object for red, one for blue, and one for black. It would make sense to use the color’s name as the global identifier. One object would have identifier “red”, one would have “blue”, and one would have “black”.

## Where Next?

The next few chapters delve into more detail on each of the topics covered in this quick start guide. If you would prefer just to see Ensembles in action, and figure out things for yourself, take a look at the Idiomatic sample app.

# The Ensemble

The Ensembles framework builds an extra concept on top of Core Data: a cluster of stores, known as an *ensemble*. It's like a musical ensemble; a hub of objects which exchange information with a single objective: to have each store reach eventual data consistency.

## What is an 'Ensemble'?

The `CDEPersistentStoreEnsemble` class is the central class of the framework. It is like an agent for your persistent store, conversing with agents on other devices that also represent stores.

An ensemble can be seen as a set of persistent stores that exchange data in order to reach eventual consistency. The `CDEPersistentStoreEnsemble` class is responsible for monitoring saves to a persistent store, exchanging this data with ensemble objects on other peers, and merging changes from other peers into its own persistent store.

You typically create one `CDEPersistentStoreEnsemble` object for each persistent store that you need to sync in your app. An ensemble has an identifier, which is used to match it with other ensemble objects on peer devices. Ensemble objects with the same identifier represent corresponding persistent stores, and the data in the persistent stores will be merged.

The process of initially setting up an ensemble object for communication with its peers is known as *leeching*. An ensemble begins in a deleeched state. Leeching prepares local storage needed by the framework, registers the device in the cloud, and migrates the data in the local persistent store into the cloud. Leeching is persistent, and typically only need be performed once, though it is possible to request that the ensemble deelech.

Once an ensemble is leeched, it can merge changes from other devices. Merging involves replaying changes from other devices, together with locally recorded changes, in order to update the persistent store. If changes are made concurrently on different devices, there is no guarantee that the data will be valid after replaying the changes. The ensemble provides delegate methods that can be used to make repairs to the data before committing. The changes made in reparation are also captured by the ensemble and transferred to other peers.

## The `CDEPersistentStoreEnsemble` Class

The `CDEPersistentStoreEnsemble` instance represents a single persistent store in the ensemble. This section goes into some of the details of using the class.

### Initializing

The `-initWithEnsembleIdentifier:persistentStoreURL:managedObjectModelURL:cloudFileSystem:` method initializes an ensemble. It stores local data in the default location, which is inside the Application Support

folder in the user's `Library`. Unless you have good reason to set the local data root elsewhere, this is the initializer you should use.

The first argument is the identifier of the ensemble. This must be the same for the ensemble objects across the syncing devices.

The absolute path to the monitored persistent store is next. The ensemble will observe saves into this store — from any context — and apply changes from other devices to the store during a merge.

A file URL for the location of the managed object model is given next. This will contain a URL for the `.mom` or `.momd` file containing the model. The reason you can't just pass the `NSManagedObjectModel` object, rather than providing a file URL, is that Ensembles needs to track past model versions, not just the most recent model.

The last parameter is the cloud file system that should be used as the backend to transfer files. This is discussed further in the next section.

The designated initializer is `-initWithEnsembleIdentifier:persistentStoreURL:managedObjectModelURL:cloudFileSystem`. Use this initializer if you need to control where the framework caches its local data. The last argument is a file URL to the directory that should be used.

## The Cloud File System

`CDECloudFileSystem` is a protocol to which classes must conform in order to act as the file syncing backend of an ensemble. The protocol has methods for testing for cloud file existence, creating directories, uploading, and downloading.

You can use one of the supplied classes (*e.g.*, `CDEICloudFileSystem` for an iCloud backend), or write your own to manage a custom backend or one not yet supported by the framework.

The cloud file system should handle all aspects of working with the backend, including authentication. If a particular `CDECloudFileSystem` class requires some user interface for logging in or performing other functions, delegate methods can be included to present the interface.

For example, the Dropbox REST API includes a modal view controller for logging in. The `CDEDropboxCloudFileSystem` invokes a delegate method when user authentication is needed, and the login controller is presented by the delegate. After logging in, a completion callback informs the cloud file system to continue. See the Idiomatic sample app for details.

## Ensemble States

The ensemble can be in a number of different states. Not all state transitions are allowed. For example, you cannot merge until the ensemble is leeches.

The `leeches` property indicates whether the ensemble is currently leeches. You should not attempt to merge unless the ensemble is in the leeches state, otherwise your merge will end with an error. In Ensembles 2, there is also a `leeching` property, which tells you if the ensemble is in the process of leeching.

The `merging` property has the value `YES` when the ensemble is in the process of merging. You probably should avoid merging when a merge is already underway, though attempting to do so will simply queue up a new merge operation, and it will run after the pre-existing merge is complete.



## Thread Safety

The Ensembles framework is not thread-safe, and you should only ever call the methods of `CDEPersistentStoreEnsemble` from the main thread.

Aside from simplifying the implementation of the framework, this policy has the added advantage of serializing access to the ensemble object, ensuring that race conditions don't arise when accessing state (eg merging property).

Most completion blocks and delegate methods are also called on the main thread. In the few cases where a background thread is used, it is documented in this book, the header file, and the Xcode documentation set.

## Queueing Tasks

The `CDEPersistentStoreEnsemble` class uses a serial queue internally for tasks like leeching and merging. If you call a method like `mergeWithCompletion:` twice, two merges will run, one after the other.

This also means you can leech and then merge as follows:

```
1 [ensemble leechWithCompletion:NULL];  
2 [ensemble mergeWithCompletion:NULL];
```

You might think this would lead to an error, since the merge method could execute before the leech is complete. In reality, the leech and merge tasks are queued, and the merge will only begin once the leech task is complete.

## Processing Pending Changes

The `processPendingChangesWithCompletion:` method can be used when you need to be sure the ensemble is fully finished any queued tasks. It flushes queued operations, and ensures all data is saved to disk.

This method is asynchronous. You know that queued tasks that exist before invoking the method have completed by the time the completion block is called. This does not guarantee that all tasks are completed if new tasks have been added in the interim.

## Discovering Ensembles

Many Core Data apps have a single persistent store, and only need a single Ensemble identifier. But document-based apps generate new stores dynamically, and need a way to detect these new stores when they are created on other devices.

The `CDEPersistentStoreEnsemble` class offers a primitive means of detecting store identifiers: the `+retrieveEnsembleIdentifiersFromCloudFileSystem:completion:` class method. This asynchronous method passes a list of ensemble identifiers found in the cloud file system to the completion block.

Even with this facility, you may be better to maintain a custom registry of document metadata (eg plist files) in a cloud directory. The metadata offers more flexibility than a single identifier string. For example, it is very likely you will want to track document name changes. The ensemble identifier is fixed — cannot be changed by the user — so recording the document name as an entry in a metadata file makes more sense.

Even if you do decide to maintain your own separate document metadata registry, Ensembles can be of help. You can use the cloud file system instance to manage the metadata file transfers.

To create a directory for the metadata in the cloud, use the `createDirectoryAtPath:completion:` method, and to upload and download the metadata files, invoke `uploadLocalFile:toPath:completion:` and `downloadFromPath:toLocalFile:completion:`, respectively. To get a list of document metadata files, use the `contentsOfDirectoryAtPath:completion:` method.

## Removing Ensembles

You can remove the cloud data of an ensemble using the class method `+removeEnsembleWithIdentifier:inCloudFileSystem:completion:`. You should use this method sparingly; only if you are sure that your devices are no longer using the ensemble.

Data removal can take some time to propagate to other devices, depending on the cloud file system. For example, with iCloud it can be many minutes or even hours before other devices update and remove the local copy of the data.

For this reason, it is not a good idea to delete the data of an ensemble, and then immediately recreate the ensemble. In a case like that, it is reasonably likely that a device will see a mix of new and old data, and enter an invalid state. This should be avoided.

Occasionally, removing the cloud data may be unavoidable, such as when a user reports some form of cloud data corruption or missing data. If this happens when using a system like iCloud, where transfer of files is asynchronous, the recommended procedure is the following:

1. Deleech the ensemble on all devices. (Deleeching removes the local cache of data.)
2. Wait 10 minutes for the devices to sync up.
3. Call the method `+removeEnsembleWithIdentifier:inCloudFileSystem:completion:` to delete the cloud data, or get the user to delete the data manually. (e.g. for iCloud you can do this in the Settings app or System Preferences)
4. Wait 10 minutes for the cloud data to disappear from all devices.
5. Leech on each device.

This procedure will generally avoid ending up with mixed sync histories, but if it turns out the ensemble can't sync, repeat the procedure, leaving even more time between steps.

## Removing Data During Development

During development you often need to start from a clean slate. Care should be taken to clean up all Ensembles data whenever the persistent store gets modified (other than by Core Data saves).

Ensembles stores data in the cloud file system, but also in a local private cache. The cache is located by default in the *Application Support* directory, in a subdirectory named the same as the app's bundle identifier. For example, the cache for the Idiomatic sample app is at `Library/Application Support/com.mentalfaculty.idiomatic.mac/com.mentalfaculty.ensembles.eventdata`. (There is an initializer for `CDEPersistentStoreEnsemble` that allows a custom location to be chosen.)

When cleaning up data during development, you can either manually remove this directory, or you can ensure that a deelech takes place, which also removes the data. Once the cloud data has been removed, and the local cache, Ensembles is effectively in a clean-slate state.

## Delegate Methods

The `CDEPersistentStoreEnsemble` fires delegate methods at various points during leeching and merging. This section briefly describes the delegate methods that you can implement. For more detail, see later chapters.

### **`persistentStoreEnsembleWillImportStore:`**

This method is invoked during leeching when the contents of the persistent store are about to be migrated to the cloud. This migration can take some time.

### **`persistentStoreEnsembleDidImportStore:`**

This is invoked during leeching when the contents of the persistent store have just been migrated to the cloud.

### **`persistentStoreEnsemble:shouldSaveMergedChanges...`**

Invoked when the ensemble is about to attempt to save merged changes into the persistent store. You can use this method to make reparations to the object graph which may have arisen due to conflicting changes. For more information, see the chapter on [Merging](#)

### **`persistentStoreEnsemble:didFailToSaveMergedChanges...`**

Invoked when the ensemble attempted to save merged changes into the persistent store, but the save failed. If you wish, you can make reparations to the object graph, and reattempt the save. For more information, see the chapter on [Merging](#).

### **`persistentStoreEnsemble:didSaveMergeChangesWithNotification:`**

This method is invoked when the persistent store has been successfully updated with changes from other devices. You can use this method to update your `NSManagedObjectContext` instances with the changes in the background save. If you don't do this, you will very likely encounter issues next time you save.

See the chapter on [Merging](#) for more information.

### **`persistentStoreEnsemble:didDeelechWithError:`**

Informs the delegate that the ensemble was forced to deelech. This should be a rare occurrence. It can be caused by the ensemble detecting some sort of data corruption, or — more commonly — the user switching cloud accounts, making the cloud data inaccessible.

See the chapter on [\[Leeching\]](#)[\[#leeching-chapter\]](#) for more information.

## **`persistentStoreEnsemble:globalIdentifiersForManagedObjects:`**

This method is used to provide global identifiers for newly created objects. You are not required to provide global identifiers, but if you do, the framework will automatically handle merging of data. Logically equivalent objects on different devices will be treated as one.

The chapter on [Models]{#models-chapter} has more detail on global identifiers.

## **Notifications**

The `CDEPersistentStoreEnsemble` class fires a number of different notifications. This section documents them all.

### **`CDEMonitoredManagedObjectContextWillSaveNotification`**

Posted when ensembles observes that a `NSManagedObjectContext` will save to the monitored persistent store. You can monitor this notification rather than the standard `NSManagedObjectContextWillSaveNotification` if you want to be sure that the ensemble has already prepared for the save when the notification is observed. If you observe `NSManagedObjectContextWillSaveNotification` directly, you can't be sure that the ensemble has observed the notification, because order of receivers is not defined.

The object for the notification is not the ensemble, but the context that is saving. The ensemble observing the save is accessible in the `userInfo` dictionary via the key `persistentStoreEnsemble`.

### **`CDEMonitoredManagedObjectContextDidSaveNotification`**

Posted when the ensemble observes that a `NSManagedObjectContext` has saved to the monitored persistent store. You can monitor this notification rather than the standard `NSManagedObjectContextDidSaveNotification` if you want to be sure that the ensemble has already processed the save when the notification is observed. If you observe `NSManagedObjectContextDidSaveNotification` directly, you can't be sure that the ensemble has observed the notification, because order of receivers is not defined.

The object for the notification is not the ensemble, but the context that is saving. The ensemble observing the save is accessible in the `userInfo` dictionary via the key `persistentStoreEnsemble`.

### **`CDEPersistentStoreEnsembleDidSaveMergeChangesNotification`**

This notification is fired after the ensemble has merged changes and performed a background save into the persistent store. You can use this notification to invoke the `mergeChangesFromContextDidSaveNotification:` method on any of the contexts that depend on the content of the store. Alternatively, you can implement the `persistentStoreEnsemble:didSaveMergeChangesWithNotification:` method for this purpose.

The object for the notification is the ensemble. The save notification, which is what is passed to the `mergeChangesFromContextDidSaveNotification:` method, is provided in the `userInfo` dictionary with the key `CDEManagedObjectContextSaveNotificationKey`.

This notification is posted on the background thread where the merge save occurred. It is important to invoke the `mergeChangesFromContextDidSaveNotification:` method on the thread/queue corresponding to the `NSManagedObjectContext` merging the changes.

## **CDEPersistentStoreEnsembleDidBeginActivityNotification**

This notification is only available in Ensembles 2. It is fired when the ensemble begins a potentially long running activity, such as leeching, deleeching, or merging.

The ensemble is the object for notification, and the type of activity is passed via the `userInfo` dictionary of the notification, for the key `CDEEnsembleActivityKey`. Activities are of the enumerated type `CDEEnsembleActivity`.

## **CDEPersistentStoreEnsembleDidMakeProgressWithActivityNotification**

This notification is only available in Ensembles 2. It is fired when the ensemble makes progress in a long running activity, such as leeching, deleeching, or merging.

The ensemble is the object for notification, and the type of activity is passed via the `userInfo` dictionary of the notification, for the key `CDEEnsembleActivityKey`. Activities are of the enumerated type `CDEEnsembleActivity`.

The overall fraction of the activity completed is stored in the `userInfo` dictionary for the key `CDEProgressFractionKey`.

## **CDEPersistentStoreEnsembleWillEndActivityNotification**

This notification is only available in Ensembles 2. It is fired when the ensemble finishes a potentially long running activity, such as leeching, deleeching, or merging.

The ensemble is the object for notification, and the type of activity is passed via the `userInfo` dictionary of the notification, for the key `CDEEnsembleActivityKey`. Activities are of the enumerated type `CDEEnsembleActivity`.

# Leeching and Deeleching

This chapter discusses how you go about coupling a persistent store with stores on other devices. This process is called *leeching*.

## Leeching

Leeching is the process of attaching the local persistent store to the ensemble.

The framework sets up some local metadata, registers the device in the cloud, and imports the local persistent store data.

The ensemble does not merge data from other devices during a leech. This will happen when you first invoke the `mergeWithCompletion:` method.

## Leeching is Persistent

Once leeched, a persistent store does not need to be leeched again, even across app relaunches. The leech state is persisted, and a store will only stop being leeched if the app specifically requests that it *deleech*, or if an event occurs that means the store cannot remain leeched.

## Leech Method

You leech a persistent store using the `CDEPersistentStoreEnsemble` method `leechPersistentStoreWithCompletion:`. This method sets up local storage and metadata, and registers the persistent store with the cloud, so that ensemble objects on other devices know of its existence.

It also converts the contents of the persistent store into transaction logs, and adds them to the cloud for merging on other devices. Because this can be a lengthy process, and can involve networking, the method is asynchronous.

If an error occurs during leeching, the ensemble will be left in a deeleched state. You will have to reattempt to leech at a later time.

You should avoid saving to the persistent store during leeching. If a save is detected, the leech will terminate with an error.

The completion block will be passed `nil` upon a successful leech, and an `NSError` otherwise.

## Delegate Methods

The `persistentStoreEnsembleWillImportStore:` delegate method is invoked during leeching when the contents of the persistent store are about to be migrated to the cloud.

The `persistentStoreEnsembleDidImportStore:` delegate method is invoked after the import is complete.

## Initial Data Migration

By default, Ensembles merges local and cloud data the first time you merge after leeching. This is what most users would expect to happen when they start syncing, but you may want to take a different approach. This section outlines different ways to approaching the initial migration of data when leeching.

### Merge Local and Cloud Data

When you leech, Ensembles automatically imports any data in the local persistent store. For most apps this is what you want. You end up with data from the local device, and data from the cloud, merged together.

The downside of this approach is that there may be circumstances where it doesn't make sense to have the data merged. For example, imagine you are syncing via iCloud, and the user temporarily switches to a different iCloud account. This will trigger Ensembles to deeleach. When it next comes time to leech again, should it merge the local and cloud data?

In this example, merging data when leeching to the second account would effectively merge the data from the two iCloud accounts, and may not be what you want. It will depend on your app and the user's mental model of how your app stores its data.

Take an address book app like Contacts. This is clearly very tightly bound to the identity of the user, and you would not want the two iCloud data sets to merge. But for other apps, which are less bound to a single identity, you may well want the data merged. For these apps, it may be very odd to the user if their data disappears when they log out of iCloud.

### Cloud Data is Definitive

If you decide your app's data set should be tightly coupled to their cloud account, and not merged when leeching, you will need to do a little preparation. Simply move aside or delete your current persistent store, and create a new, empty one, before leeching. This way, when you leech the store, you will only end up with data that is currently in the cloud.

This model of data migration is the approach adopted in Core Data's native sync via iCloud. If you want to closely approximate Apple's approach, this is how you should go about it.

Always favoring the cloud data works well for app's that can assume they will always be used with sync active, but you generally also have to consider the case that the user may not be logged in to the cloud service. A standard way to handle this is to keep a separate *fallback* store around which is used when logged out of the cloud. This store will have different data to the cloud store, but at least your app will be capable of working without the cloud, in purely local mode.

Whenever you move or delete a persistent store, it is important to include all files that the store depends on. With the recent introduction of SQLite journaling to Core Data, each store corresponds to at least three different files: the main database, and files with the extension `.db-wal` and `.db-shm`. Additionally, any external binaries are stored in the hidden `.<Store Name>_SUPPORT` directory.

## Switch Between Local and Cloud Data

The last option commonly used in apps is to offer the user a choice of which data to keep when the app leeches. The user could opt to keep only the cloud data, or they could opt to keep only the local data.

Keeping only the local data requires existing cloud data to be deleted before leeching. You can use the `CDEPersistentStoreEnsemble` class method `removeEnsembleWithIdentifier:inCloudFileSystem:completion:` for this purpose.

That said, you should be very careful when removing cloud data. For cloud services where changes take time to propagate between devices, such as iCloud, it is usually ill-advised. The reason is that deleting data on one device may lead to other devices temporarily having a mix of files from different sync histories, and this can corrupt the ensemble completely.

The only time when you can safely consider deletion of cloud data before leeching is when you can be sure that it is an atomic operation. This is the case for services with a central server and no local file cache, such as Dropbox (REST API) and WebDAV.

If you would still like to support this option with a service like iCloud, it is best to inform your user to delete the data on all devices before attempting to perform a new sync (*i.e.* leech). This should ensure that there are no stray files corrupting the new ensemble.

## Leeching an Existing App for the First Time

If you are adding Ensembles to an existing app, it is likely that you will need to perform a version migration in order to add global identifiers to the model. It is important that when this happens, you add and save the global identifiers *before* trying to the leech for the first time. If you don't, it is possible that Ensembles will inadvertently cause global identifiers to be generated when importing the store data, and the identifiers stored in Ensembles may not correspond to the ones used in your main context.

How you detect whether global identifiers are needed is somewhat app specific. You could determine whether a migration is needed, and run code to update global identifiers whenever that happens.

```

1  NSDictionary *metadata = [NSPersistentStoreCoordinator metadataForPersistentStoreOfType:NS\
2  SQLiteStoreType
3      URL:storeURL error:NULL];
4  NSManagedObjectModel *currentModel = persistentStoreCoordinator.managedObjectModel;
5  BOOL migrationNeeded = ![currentModel isConfiguration:nil compatibleWithStoreMetadata:meta\
6  data];
7
8  // Migrate store, usually by simply adding it to the persistent store coordinator
9  ...
10
11 if (migrationNeeded) [self updateGlobalIdentifiers];

```

The `updateGlobalIdentifiers` method would fetch any objects that don't have global identifiers assigned, assign them, and save. A simple, generic implementation would just loop over entities, setting global identifiers to a UUID.



```

1 - (void)updateGlobalIdentifiers
2 {
3     for (NSEntityDescription *entity in managedObjectModel) {
4         NSFetchRequest *fetch = [NSFetchRequest fetchRequestWithEntityName:entity.name];
5         fetch.predicate = [NSPredicate predicateWithFormat:@"uniqueIdentifier = NIL"];
6         NSArray *objects = [managedObjectContext executeFetchRequest:fetch error:NULL];
7         for (NSManagedObject *object in objects) {
8             if ([object valueForKey:@"uniqueIdentifier"]) continue;
9             NSString *newID = [[NSUUID UUID] UUIDString];
10            [object setValue:newID forKey:@"uniqueIdentifier"];
11        }
12        [managedObjectContext save:NULL];
13        [managedObjectContext reset];
14    }
15 }

```

A more advanced approach might include a method for setting the unique identifier in the `NSManagedObject` subclasses (e.g. `setGlobalIdentifierIfNeeded`), and invoke it on the fetched objects.

Another way to test whether global identifiers need to be initialized, which avoids checking the managed object model, is to perform a fetch for the count of objects that are missing a global identifier. If global identifiers are missing, they need to be added.

```

1 NSFetchRequest *fetch = [NSFetchRequest fetchRequestWithEntityName:@"MainEntity"];
2 fetch.predicate = [NSPredicate predicateWithFormat:@"uniqueIdentifier = NIL"];
3 NSUInteger count = [managedObjectContext countForFetchRequest:fetch error:NULL];
4 if (count > 0) [self updateGlobalIdentifiers];

```

As a general rule, it is best not to set global identifiers in `awakeFromFetch:`, as this may leave the store in an inconsistent state, where some objects have global identifiers — the ones that have been fetched — and others don't. Set global identifiers for new objects in `awakeFromInsert:`, and handle legacy stores as described above in order to introduce global identifiers.

## Deeleaching

Detaching a store from an ensemble is known as *deeleaching*. The framework removes local metadata and transaction logs, and will no longer monitor saves or merge results from other stores.

### Deeleach Method

To deeleach a store, you can invoke the `deeleachPersistentStoreWithCompletion:` method of `CDEPersistentStoreEnsemble`.

The completion block is executed when deeleaching completes. The block is passed `nil` upon a success, and an `NSError` otherwise.

## Forced Deeleching

In normal operation, an ensemble will not deelech unless requested to do so. However, circumstances can arise that can compromise the integrity of the sync data, in which case, the ensemble can elect to spontaneously deelech.

This is usually caused by a cloud account switch, which causes cloud data to become inaccessible, but can also be caused by the framework detecting some data tampering or corruption.

When a forced deelech occurs, the `persistentStoreEnsemble:didDeelechWithError:` delegate method is invoked. You can use this method to update the interface to reflect the deeleched state, and perhaps notify the user of the issue. You can also attempt to leech again in order to continue syncing.

# Merging

Merging involves retrieving new data from other devices, combining it with recent local changes, and updating the persistent store. It is what would typically be considered a *sync* operation.

This chapter describes the process of merging in some detail, and discusses various strategies for when to perform a merge.

## What Happens During a Merge?

A single merge actually involves many smaller steps. We'll go through these steps in this section.

### Importing new Transaction Logs

Ensembles first checks for new files in the cloud. These get downloaded, and then imported into the *Event Store*, which is a local cache for the transaction logs.

### Consolidating Baselines

A baseline represents an initial state for the store. It is the set of all insertions needed to fully form the store.

Usually there is only one baseline. To get the current persistent store, you would playback all the changes in the baseline, and then play back any extra transaction logs that have appeared since the baseline was formed.

Sometimes there can be more than one baseline. For example, this occurs when a new device joins the ensemble, and uploads the contents of its local store. When this happens, you end up with the original baseline, plus the newly uploaded baseline.

To get back to having just one baseline, Ensembles needs to consolidate the two baselines into one. Sometimes this is straightforward, such as when one baseline represents an earlier version of the other. In a case like this, the most recent baseline is kept, and the other discarded.

In other cases, the two baselines are unrelated, and need to be merged. This involves going through all the changes in each baseline and picking the most recent for each object. Effectively, we are taking the union of the two baselines. Where there is overlap, changes get merged.

After consolidating baselines, there should be only one baseline again.

### Rebasing

After consolidation, we know there is only one baseline, but it might not be very recent. If it is too old, there may be a lot of new transaction logs, and a lot of data redundancy. This results in excessive cloud storage, and less efficient performance, because the framework has to process the data even if it is irrelevant.

Rebasing involves propagating the existing baseline forward in time, coalescing transactions, in order to form a new baseline.

Ensembles uses rough heuristics to estimate whether it is worthwhile to rebase. If there is a saving in cloud data storage of 50% or more to be had, it will rebase. It will also rebase if the number of cloud files is getting excessive.

## Consistency Checks

Ensembles tracks the versions of every transaction log file, so it can detect when there are files missing. After the rebasing stage, a number of checks are carried out to ensure there are no important log files missing. If there are some missing log files, an error will be issued, and the merge stopped, to allow the missing files to be transferred.

## Replaying Changes

Once it has been determined that all data is present, Ensembles determines what transaction log files are new since the last merge. If there are some from other devices, it gathers together the new changes, as well as any logs that occurred concurrently with those files. The logs are then ordered, and played back to update the persistent store.

When applying the updates, Ensembles does not make use of any objects from the app's main Core Data stack. Instead, it sets up a completely separate stack, which only shares the persistent store file itself. It makes all changes in this private stack.

If at some point during this process the app saves to the persistent store, the merge will be terminated with an error. It can simply be retried at a later time.

## Committing to the Persistent Store

Conflicting changes on different devices mean that after all new transaction logs have been replayed, the object graph in the private merge context may not be in a valid state. The `CDEPersistentStoreEnsemble` gives its delegate an opportunity to check for and correct this.

First, it invokes `persistentStoreEnsemble:shouldSaveMergedChangesInManagedObjectContext:reparationManagedObjectContext:`. This method allows you to make updates before an attempt is made to save the changes to the persistent store. It also delineates the point-of-no-return for the merge, giving you a chance to terminate by returning `NO`.

This method is called on a background thread, and each context passed has private-queue concurrency, and should be accessed as such with either `performBlock:` or `performBlockAndWait:`. The two contexts also have a parent-child relationship, so it is unwise to nest calls to `performBlock:`, which would likely result in deadlock.

You can access what has been changed in the merge via the first context. Use the standard `NSManagedObjectContext` properties `insertedObjects`, `updatedObjects`, and `deletedObjects`.

If you actually want to make repairs, *do not make them in the first context*. Instead, use the reparation context. This will require you to pass `NSManagedObjectContext` IDs between blocks, but it is for good reason: the reparation context will track any changes you make, and these will be added to the transaction logs and applied on other devices.

If you return YES, the ensemble will attempt to save the merge changes to the persistent store. If the save fails, the delegate method `CDEPersistentStoreEnsemble:didFailToSaveMergedChangesInManagedObjectContext:error:reparationContext:` gets invoked. The error that arose in saving is passed in, and you can use it to determine what went wrong. If you would like to attempt to fix the error, you can again use the reparation context to make changes, and return YES to request a retry.

More detail on conflict resolution and repairs, including sample code, is provided in a [later chapter](#).

## Notifying of Commit

If saving to the persistent store succeeds, the `CDEPersistentStoreEnsemble:didSaveMergeChangesWithNotification:` delegate method is called. This is a good point to merge changes into any contexts that depend on the store. Invoke the `mergeChangesFromContextDidSaveNotification:` method of each context, and be careful to do this on the appropriate thread or queue.

## Exporting New Transaction Logs

At this point, the framework exports any new transaction logs to file, and uploads them to the cloud for other devices to import. It also cleans up any files that are no longer needed in the cloud.

## Triggering Merges

Ensembles does not trigger merges automatically — all merges are initiated by your application code. This is quite deliberate, because every app is different, and each will demand its own approach to when and how often merges take place. By requiring the app code to trigger merges, Ensembles offers you maximum control over sync operations.

## Initiating a Merge

You begin merging by invoking `mergeWithCompletion:` on the `CDEPersistentStoreEnsemble`. The method is asynchronous, because merging involves networking, and other long running tasks.

The completion block is called back on the main thread. It includes a single `NSError` as solitary parameter; this will be `nil` upon successful completion.

A merge can fail for a variety of reasons, from file downloads being incomplete, to the merge being interrupted by a save to the persistent store. Errors during merging are not typically very serious, and you should just retry the merge a bit later. Error codes can be found [later](#) in this chapter.

## Progress of a Merge

There is no mechanism for following the progress of the merge process in Ensembles 1.x, but Ensembles 2 fires `CDEPersistentStoreEnsembleDidMakeProgressWithActivityNotification` notifications when progress is made. By retrieving the value corresponding to `CDEProgressFractionKey` in the user info dictionary, you can use the notifications to update a progress indicator.

The completion block also tells you the merge has completed or failed, and is an appropriate place to update the user interface to show that the sync has finished.

## Observing Merges

You can test whether an ensemble is currently merging using the `isMerging` boolean property. You can also use KVO to observe changes to this property, and, *e.g.*, show an activity indicator while merging is taking place.

## Canceling a Merge

If you decide you want to cancel a merge, invoke the `cancelMergeWithCompletion:` method, which is asynchronous. The completion will be called when the merge completes, or terminates prematurely.

## When to Trigger Merges

There are some common times when most apps will want to merge changes, such as on launch and when terminating. Some apps will also want to merge at other times, perhaps at regular time intervals, or when the user taps a *Sync Now* button. The next few sections describe various strategies for initiating merges.

### Launching

When the application first launches, it is often a good moment to initiate a merge. There will usually be new transaction logs from other devices that need to be imported, and a merge will give the user confidence that everything is working properly.

The `application:didFinishLaunchingWithOptions:` method on iOS, and `applicationDidFinishLaunching:` on the Mac, are good places to trigger the merge.

### Terminating

On the Mac, when your app quits you will likely want to trigger a merge.<sup>45</sup> This will export new transaction logs and make them available to other devices. If you don't do this, your users may be frustrated that changes they make on one device do not appear later when they open your app on another device.

It is also important to allow Ensembles to save its internal data to disk before terminating, so even if you don't merge, you will need to cleanly handle termination on the Mac. You can postpone termination by returning `NSTerminateLater` from the `applicationShouldTerminate:` app delegate method, and then invoking `replyToApplicationShouldTerminate:` on the `NSApplication` when the operation is complete.

---

<sup>45</sup> On iOS, you are more likely to merge when the app enters the background.

```
1 -(NSApplicationTerminateReply)applicationShouldTerminate:(NSApplication *)sender
2 {
3     [managedObjectContext save:NULL];
4
5     [persistentStoreEnsemble processPendingChangesWithCompletion:^(NSError *error) {
6         [[NSApplication sharedApplication] replyToApplicationShouldTerminate:YES];
7     }];
8
9     return NSTerminateLater;
10 }
```

The `processPendingChangesWithCompletion:` method makes sure that changes arising from the save are fully processed and saved to disk.

If you would like other devices to get the latest changes, you will need to merge when terminating instead.

```
1 -(NSApplicationTerminateReply)applicationShouldTerminate:(NSApplication *)sender
2 {
3     [managedObjectContext save:NULL];
4
5     [persistentStoreEnsemble mergeWithCompletion:^(NSError *error) {
6         [[NSApplication sharedApplication] replyToApplicationShouldTerminate:YES];
7     }];
8
9     return NSTerminateLater;
10 }
```

Note that a merge can take some time, particularly if the cloud file system must perform networking. You will probably want to display a progress indicator to the user while this takes place.

## Becoming Active or Inactive

Another good time for a Mac app to merge is when it becomes active or inactive. The user may stop using the app, but leave it running in the background. You will probably want to save and merge as the app becomes inactive, so if it remains inactive for some time, at least the latest changes will appear on other devices.

It is also wise to merge when the app becomes active again, to pull in changes from other devices, which the user will expect to appear.

You can use the app delegate methods `applicationDidBecomeActive:` and `applicationWillResignActive:` to invoke the `mergeWithCompletion:` method.

## Entering the Background

On iOS, it is generally best to merge when the app enters the background. To ensure the app continues to execute while the merge is taking place, you can register a background task before merging, and end the task when the merge completes.

You would typically use the `applicationDidEnterBackground:` app delegate method for this purpose.

```

1 - (void)applicationDidEnterBackground:(UIApplication *)application
2 {
3     UIBackgroundTaskIdentifier identifier = [[UIApplication sharedApplication]
4         beginBackgroundTaskWithExpirationHandler:NULL];
5
6     dispatch_async(dispatch_get_global_queue(0, 0), ^{
7         [managedObjectContext performBlock:^(
8             if (managedObjectContext.hasChanges) {
9                 [managedObjectContext save:NULL];
10            }
11
12            [persistentStoreEnsemble mergeWithCompletion:^(NSError *error) {
13                [[UIApplication sharedApplication] endBackgroundTask:identifier];
14            }]];
15        });
16    });
17 }

```

Changes in the main context are first saved in order to commit them to the store, and form transaction logs. Once the context is saved, you can initiate a merge, and end the background task in the completion handler.

## Entering the Foreground

An iOS app may remain in the background for some time. In order to ensure the user gets the latest data from other devices when they next use the app, you will want to merge in `applicationWillEnterForeground:`.

## Notifications

Some backends<sup>46</sup> fire notifications when new data from other devices is detected. This is usually a good time to perform a merge.

For example, the `CDEICloudFileSystem` fires the notification `CDEICloudFileSystemDidDownloadFilesNotification` when new file downloads complete. You could set up an observer for this notification, and invoke `mergeWithCompletion:` whenever it fires.

Merging as soon as data is detected makes sync feel snappy, and will be appreciated by your users.

## Timers

If you don't have a backend that conveniently notifies you when new data is available, but you do want to keep store data in close sync, you could setup a timer to intermittently invoke the `mergeWithCompletion:` method. For example, here is a timer that fires every two minutes.

---

<sup>46</sup>Classes conforming to the `CDEICloudFileSystem` protocol.



```

1 timer = [NSTimer scheduledTimerWithTimeInterval:120.0 target:self selector:@selector(perfo\
2 rmScheduledMerge:)
3         userInfo:nil repeats:YES];

```

The `performScheduledMerge:` method would simply invoke `mergeWithCompletion:` on the ensemble.

```

1 - (void)performScheduledMerge:(NSTimer *)aTimer
2 {
3     [ensemble mergeWithCompletion:NULL];
4 }

```

Don't forget to invalidate the `NSTimer` when you no longer need it to fire.

```

1 [timer invalidate];

```

## Manual Trigger

You may want to offer your users the option of manually triggering a merge. This doesn't have to be a major feature of the user interface; it could be an extra button hidden away with the sync settings.

Having a button that immediately triggers a sync gives your users a feeling of control, even if most of the time they won't need it.

## Using Background Fetch Mode

iOS allows apps to utilize background modes to carry out limited tasks while the app is not active. It is not advisable to perform a full merge using background modes, because you have no control over how long the merge will take, but you might be able to use the background fetch mode to download files or request them such that they are ready to merge when the app enters the foreground.

For example, the iCloud backend automatically requests new files when it observes metadata notifications. If you activate the background fetch mode for your app, you could implement background fetching by first setting the fetch interval

```

1 [[UIApplication sharedApplication] setMinimumBackgroundFetchInterval:UIApplicationBackgrou\
2 ndFetchIntervalMinimum];

```

and then including an application delegate method like this

```

1 -(void)application:(UIApplication *)application
2   performFetchWithCompletionHandler:(void (^)(UIBackgroundFetchResult))completionHandler
3 {
4     if (!ensemble.isLeeched) {
5         completionHandler(UIBackgroundFetchResultNoData);
6     }
7     else {
8         dispatch_async(dispatch_get_main_queue(), ^{
9             CDEICloudFileSystem *cloudFileSystem = (id)ensemble.cloudFileSystem;
10            completionHandler(cloudFileSystem.bytesRemainingToDownload > 0 ?
11                UIBackgroundFetchResultNewData : UIBackgroundFetchResultNoData);
12        });
13    }
14 }

```

## Updating Contexts

Ensembles merges changes from other devices into a private background context before saving to the persistent store. When this happens, it is important to inform the contexts in your control of the merge. This section describes how.

### Merging Sync Changes into a Context

If you are using a single `NSManagedObjectContext` in your app, you should implement the `persistentStoreEnsemble:didSaveMergeChangesWithNotification:` method — or observe the corresponding notification — in order to inform your context of a background sync.

```

1 - (void)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble
2   didSaveMergeChangesWithNotification:(NSNotification *)notification
3 {
4     [managedObjectContext performBlock:^(
5         [managedObjectContext mergeChangesFromContextDidSaveNotification:notification];
6     )];
7 }

```

The method is invoked on a background thread; make sure you shunt execution to your app's context thread or queue before invoking `mergeChangesFromContextDidSaveNotification:`.

### Multiple Contexts

It is not unusual multiple `NSManagedObjectContext`s in your app. Some have a private-queue parent context for saving to the persistent store, with the main thread context a child context. Other apps may use secondary contexts for importing data and other tasks.

When Ensembles merges, it is important to refresh *all* of the contexts that are in any way related to the affected persistent store. So the delegate method may look something like this.

```

1 - (void)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble
2   didSaveMergeChangesWithNotification:(NSNotification *)notification
3 {
4     [backgroundQueueContext performBlockAndWait:^(
5         [backgroundQueueContext mergeChangesFromContextDidSaveNotification:notification];
6     )];
7
8     [mainContext performBlockAndWait:^(
9         [mainContext mergeChangesFromContextDidSaveNotification:notification];
10    )];
11
12    [importContext performBlockAndWait:^(
13        [importContext mergeChangesFromContextDidSaveNotification:notification];
14    )];
15 }

```

## Delegate Methods

The `CDEPersistentStoreEnsemble` class includes a number of delegate methods that are invoked during a merge. They are described here in detail.

### The Should-Save-Merge-Changes Method

The `persistentStoreEnsemble:shouldSaveMergedChangesInManagedObjectContext:reparationManagedObjectContext:` delegate method is invoked when the ensemble is about to attempt to save merged changes into the persistent store.

This method is invoked on a background thread. Both of the contexts passed have private queue concurrency type, and so they should only be accessed via calls to `performBlock...` methods.

You can use the saving context to check what changes have been made in the merge via `NSManagedObjectContext` methods like `insertedObjects`, `updatedObjects`, and `deletedObjects`.

You should not make any changes directly in the saving context. If you need to make changes before the save is attempted, you can make them in the reparation context.

You can force the merge to terminate altogether by returning `NO` from this method.

Be careful not to nest calls to the `performBlock...` methods for the two contexts. This will very likely lead to a deadlock, because the contexts in question have a parent-child relationship.

### The Did-Fail-To-Save-Merged-Changes Method

The `persistentStoreEnsemble:didFailToSaveMergedChangesInManagedObjectContext:error:reparationManagedObjectContext:` method gets invoked when the ensemble attempted to save merged changes into the persistent store, but the save failed.

This method is invoked on a background thread. Both of the contexts passed have private queue concurrency type, and so they should only be accessed via calls to `performBlock...` methods.

You can use the saving context to check what changes failed to save via `NSManagedObjectContext` methods like `insertedObjects`, `updatedObjects`, and `deletedObjects`.

The error that occurred during saving is passed and can be used to determine which objects are responsible for the failure. One possible implementation of the delegate method might look as follows.

```

1  -(BOOL)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble
2      didFailToSaveMergedChangesInManagedObjectContext:(NSManagedObjectContext *)savingContext
3  {
4      NSError *error;
5      NSManagedObjectContext *reparationContext;
6      {
7          NSMutableArray *objectIDs = [NSMutableArray array];
8          NSMutableArray *errors = [NSMutableArray array];
9
10         [savingContext performBlockAndWait:^(
11             if ( error.code != NSValidationMultipleErrorsError ) {
12                 NSManagedObject *object = error.userInfo[@"NSValidationErrorObject"];
13                 [objectIDs addObject:object.objectID];
14                 [errors addObject:error];
15             }
16             else {
17                 NSArray *detailedErrors = error.userInfo[NSDetailedErrorsKey];
18                 for ( NSError *error in detailedErrors ) {
19                     NSDictionary *detailedInfo = error.userInfo;
20                     NSManagedObject *object = detailedInfo[@"NSValidationErrorObject"];
21                     [objectIDs addObject:object.objectID];
22                     [errors addObject:error];
23                 }
24             }
25         }];
26
27         [reparationContext performBlockAndWait:^(
28             [objectIDs enumerateObjectsWithOptions:0
29                 usingBlock:^(NSManagedObjectID *objectID, NSUInteger index, BOOL *stop) {
30                     NSError *error;
31                     id object = [reparationContext existingObjectWithID:objectID error:&error];
32                     if (!object) {
33                         NSLog(@"Failed to retrieve invalid object: %@", error);
34                         return;
35                     }
36                     [object repairWithError:errors[index]];
37                 }];

```

```

38     }];
39
40     return YES;
41 }

```

This code assumes that you are using a `NSManagedObject` subclass with a `repairWithError:` method that can handle validation errors. The following example of one such method, which handles validation errors caused by orphaned objects by simply deleting the object.

```

1 -(void)repairForError:(NSError *)error
2 {
3     if ( error.code == NSValidationMissingMandatoryPropertyError ||
4         error.code == NSManagedObjectValidationError ||
5         error.code == NSValidationRelationshipLacksMinimumCountError ) {
6         [self.managedObjectContext deleteObject:self];
7     }
8 }

```

You should not make any changes directly in the saving context. If you wish to reattempt the save, make any necessary changes in the reparation context, and then return YES.

You can force the merge to terminate altogether by returning NO from this method.

## The Did-Save-Merge-Changes Method

The `persistentStoreEnsemble:didSaveMergeChangesWithNotification:` delegate method is invoked after the ensemble successfully saves merged changes into the persistent store.

This method is invoked on a background thread. The notification passed includes a `userInfo` dictionary with the `NSManagedObjectID` instances for objects that were changed in the merge. It can be used to determine what insertions, updates, and deletions occurred.

You will usually want to pass this notification to the `mergeChangesFromContextDidSaveNotification:` method of any context that accesses the persistent store, be it directly or indirectly. This will allow the contexts to account for the changes.

As always, be sure to invoke the `mergeChangesFromContextDidSaveNotification:` method on the thread or queue corresponding to the messaged context.

## Merge Errors

Errors during merging are common, and generally not serious. You should treat them as you would networking errors. The merge may have failed, but usually it is just a question of retrying later.

You can find all error codes in the header file *CDEDefines.h*.

## Common error codes that can arise during a merge

| Error Code                          | Description                                                                                                               |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| CDEErrorCodeCancelled               | The operation was cancelled.                                                                                              |
| CDEErrorCodeDisallowedStateChange   | Request for invalid state transition was made. <i>Eg</i> Merge during leeching.                                           |
| CDEErrorCodeFileCoordinatorTimedOut | Accessing a file with a coordinator timed out. Often because iCloud is still downloading. Retry later.                    |
| CDEErrorCodeFileAccessFailed        | An attempt to access a file failed.                                                                                       |
| CDEErrorCodeDiscontinuousRevisions  | Some change sets are missing. Usually temporarily missing data. Retry a bit later.                                        |
| CDEErrorCodeMissingDependencies     | Some change sets are missing. Usually temporarily missing data. Retry a bit later.                                        |
| CDEErrorCodeCloudIdentityChanged    | User changed cloud identity. This forces a deleech.                                                                       |
| CDEErrorCodeDataCorruptionDetected  | Some left over, incomplete data has been found. Probably due to a crash.                                                  |
| CDEErrorCodeUnknownModelVersion     | A model version exists in the cloud that is unknown. Merge will succeed again after update that includes the new version. |
| CDEErrorCodeStoreUnregistered       | The ensemble is no longer registered in the cloud. Usually due to cloud data removal.                                     |
| CDEErrorCodeSaveOccurredDuringMerge | A save to the persistent store occurred during merge. You can simply retry the merge.                                     |
| CDEErrorCodeMissingStore            | There is no persistent store at the path. Ensure a store exists and try again.                                            |
| CDEErrorCodeMissingDataFiles        | Files used to store large NSData attributes are missing. Usually temporary. Retry a bit later.                            |
| CDEErrorCodeNetworkError            | A generic networking error occurred.                                                                                      |
| CDEErrorCodeServerError             | An error from a server was received.                                                                                      |
| CDEErrorCodeConnectionError         | The cloud file system could not connect.                                                                                  |
| CDEErrorCodeAuthenticationFailure   | The user failed to authenticate.                                                                                          |

# Saving

When your app saves data, Ensembles records the changes it makes to the persistent store. This chapter is about how Ensembles stores the changes, the impact that has on your app's execution, and what you should be mindful of during saving.

## Monitoring Saves

Ensembles tracks changes to your Core Data persistent store by observing all save operations that modify the store — regardless of the context that is saving — and recording the changes made. The changes are stored locally, and transferred to other devices during the next merge, where they are 'played back' to make comparable changes to the other stores in the ensemble.

## Observing Changes to the Persistent Store

When any save occurs, in any `NSManagedObjectContext`, the Ensembles framework observes the `NSManagedObjectContextWillSaveNotification` notification, and ascertains whether the saving context will directly modify the persistent store that it is monitoring. If the context is saving directly into the store, the save will be handled by the framework.

If the context does not save directly into the persistent store, it will be ignored. For example, it is common to save data to the persistent store via a background context, which is a parent to the main context. In this scenario, only the saves of the background context will be handled — saves of the main context will be ignored.

If the framework determines that the context firing `NSManagedObjectContextWillSaveNotification` will modify the persistent store, it stores some pre-save data that is needed later when deducing what changes were made. In particular, there is not enough information post-save to determine what changes were made in to-many relationships, so the state of these relationships is captured pre-save.

Upon observing the corresponding `NSManagedObjectContextDidSaveNotification`, Ensembles generates change objects or *deltas*. The properties of inserted and updated objects are stored, together with a list of deleted objects, in a cache known as the *event store*. It also stores ordering information, such as a timestamp, which is used during merging.

The saved changes do not become available to other devices until the next merge operation, when they get exported to transaction log files, and uploaded to the cloud.

## Notifications Fired During Saves

If your app carries out tasks when observing save notifications, it may be important to ensure that these tasks occur at the right time in relation to Ensembles handling of the same notifications. Because the order of notification receivers is not defined, the Ensembles framework fires extra notifications that can be used in place of the standard notifications in order to guarantee correct ordering.

The `CDEMonitoredManagedObjectContextWillSaveNotification` notification is fired when Ensembles has ascertained that the saving context will modify the persistent store, but before it has stored any values in preparation for the save.

The `CDEMonitoredManagedObjectContextDidSaveNotification` is fired after the ensemble has observed `NSManagedObjectContextDidSaveNotification` and has finished saving the changes to the event store.

In both cases, the notification object is the saving context. The ensemble object is passed via the `userInfo` dictionary.

## The Cost of Saving Changes

In order to sync, Ensembles must store all of the data in your persistent store as change sets (deltas). This has a cost, and is unavoidable. Effectively, data has to be saved twice, in two different forms.

Most of the cost of saving changes is incurred when Ensembles receives the `NSManagedObjectContextDidSaveNotification`. It goes through the updated values, deleted objects, and insertions, and generates objects to represent these changes in the event store.

You should keep in mind that save operations will be more expensive when using Ensembles than they would normally be. For most apps, it will have negligible impact, but you should be aware of the performance cost of saving.

## Committing Changes

Ensembles doesn't register changes to your objects until they are saved. This can raise issues for certain kinds of data models. This section covers the implications of these issues, and strategies to deal with them.

### Changes Don't Exist Until Saved

It is important to stress that Ensembles only registers changes when they are saved to the persistent store. Before that, the changes may exist in memory, but Ensembles has no 'knowledge' of their existence. If you are not mindful of this, you can end up with some perplexing bugs.

### Save-Merge Race Conditions

One possibility is that a race condition can arise between saving newly inserted objects, and merging in logically-equivalent objects from another device. This is not a general issue, because usually your objects will have a random global identifier, but if you have objects that have carefully chosen identifiers, and objects with the same identifier could be inserted on multiple devices, you need to be wary of this risk.

As an example, imagine you have a tag-like class where logically-equivalent objects can be inserted on different devices. One device could insert an object for the tag 'core-data', and a corresponding object could be created with the same tag on another device at about the same time. Usually, you would use the tag string as the global identifier, and expect that there should never be more than one object on each device for each string.



Ensembles can handle this scenario, enforcing uniqueness of the tag objects, as long as it is aware of all the relevant change sets. Unfortunately, there is a window of time where an object exists in memory, but has not been saved, and is thus unknown to the framework. It is possible that you create an object with the tag ‘core-data’, but do not save it, and while it is unsaved, Ensembles merges changes from another device, and creates a new object with the same tag. You could end up with two objects each with the tag ‘core-data’, *ie*, two objects sharing the same global identifier.

This is obviously undesirable. It doesn’t apply to all apps, but may apply to yours. Luckily, there are ways to avoid it, such as saving promptly, and avoiding merges when there are unsaved changes. These strategies are discussed later in the chapter.

## Accidental Duplicates

Another risk is that two objects exist with the same global identifier in the same save operation, even if the two objects never co-exist in memory.

This probably sounds like it shouldn’t be possible, but consider the following example: You have an object with global identifier ‘123’. You delete the object, and — before saving — you insert a new object with the global identifier ‘123’.

It would seem like this should be OK, because there is only ever one object with the identifier ‘123’ in memory at any point in time. But from the perspective of the Ensembles framework, it only sees these changes when you next save. It will see a deleted object with global identifier ‘123’, and a different, inserted object with the same identifier.

The solution to this is simply to commit your changes when you make them, by saving. Save after deleting the object, and then insert the new object. In this way, Ensembles can properly order and handle the events.

## Approaches to Saving

Some apps are built on a single context, with simple, synchronous saves, and others use more complex Core Data stacks and approaches to saving data. This section covers common approaches, and special considerations for each as they relate to Ensembles.

### Synchronous Saving

Many apps are built upon a single, main thread `NSManagedObjectContext`, and saves occur synchronously. This has an advantage when it comes to Ensembles, because you know exactly when your changes are committed to the persistent store, and thus when the framework is made aware of the changes. There are no race conditions: when the `save:` method returns, the changes have been made to the persistent store, and will be committed to the Ensembles’ event store.

### Asynchronous Saving

With the introduction of nested contexts in Core Data, it has become common practice to use a background, private queue context for committing changes to the persistent store. The main queue context used by the

user interface is a child of the background context, and when it saves, it pushes changes up to the background context, but does not modify the persistent store.

The advantage of this approach is that the main queue context doesn't need to perform disk IO, making saving much faster. This is important on the main thread, because any significant pause will block the user interface.

With nested contexts, an app usually saves in two steps. First, the main queue context saves changes, pushing them up to the background context. Sometime later, the background context will save on a background queue, committing changes to the persistent store. It is this later save that Ensembles will monitor and handle.

In many apps, this will work fine with Ensembles, but in some cases, a race condition can arise. This can occur if logically-equivalent objects — objects with the same global identifier — can be inserted on more than one device. In this scenario, which was introduced earlier in the chapter, an unsaved object could exist in memory, while another object with the same global identifier is merged into the store by Ensembles.

When the potential exists in your data model for this race condition to arise, there are a number of measures you can take to avoid it. The next few sections discuss these measures.

## Use Synchronous Saves to Avoid Race Conditions

The most straightforward way to avoid a race condition leading to duplicate objects is to save your contexts as soon as susceptible objects are inserted, and to do so synchronously. This reduces the window for a race condition.

The worst thing you can do is insert objects and wait several minutes or more before saving. Doing so makes the likelihood of duplicate objects much greater.

As an example, imagine you have an app where the main context is a child of a private-queue background context, which saves into the persistent store. If you insert an object that is susceptible to the race condition, such as the tag-like objects discussed earlier, you would immediately save the main context *and* the background context.

```
1 [mainContext performBlockAndWait:^(
2     [mainContext save:NULL];
3
4     [mainContext.parentContext performBlockAndWait:^(
5         [mainContext.parentContext save:NULL];
6     }]);
7 ]];
```

This may partially reduce the benefit of having the background context in the first place, but hopefully there are still some save operations that can be performed asynchronously and will still benefit.

## Avoiding Merges with Unsaved Changes

Ensembles will cancel any merge operation that happens to be active when your app performs a save to the persistent store, so using synchronous saves makes it unlikely that a merge-save race condition will lead to duplicate objects. You can make it even less likely by actively cancelling merges when there are unsaved changes, using a `CDEPersistentStoreEnsemble` delegate method.

```

1 - (BOOL)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble
2     shouldSaveMergedChangesInManagedObjectContext:(NSManagedObjectContext *)savingContext
3     reparationManagedObjectContext:(NSManagedObjectContext *)reparationContext
4 {
5     return !(mainContext.hasChanges || mainContext.parentContext.hasChanges);
6 }

```

This method is invoked just before the merge will update the persistent store. Returning NO causes the merge to cancel before the store is modified.

## Termination

It is important to realize that saving your data with Ensembles is an asynchronous process. The save to the main context is synchronous, as always with Core Data, but Ensembles continues on in the background, writing the changes to a local cache so they can be sent to other devices. If your app terminates (quits) before giving Ensembles the opportunity to commit its changes to disk, Ensembles may be forced to deelech to ensure data integrity.

## Terminating on iOS

Usually, you save data in the `applicationDidEnterBackground:` method on iOS, but often also in `applicationWillTerminate:`. In both places, you should — at the very least — use the `processPendingChangesWithCompletion:` method of `CDEPersistentStoreEnsemble` to give Ensembles a chance to complete writing its changes to disk.

```

1 - (void)applicationDidEnterBackground:(UIApplication *)application
2 {
3     UIBackgroundTaskIdentifier identifier =
4         [[UIApplication sharedApplication] beginBackgroundTaskWithExpirationHandler:NULL];
5     dispatch_async(dispatch_get_global_queue(0, 0), ^{
6         [managedObjectContext performBlock:^(
7             [managedObjectContext save:NULL];
8             [ensemble processPendingChangesWithCompletion:^(NSError *error) {
9                 [[UIApplication sharedApplication] endBackgroundTask:identifier];
10            }];
11        }]);
12    });
13 }

```

Note that a background task is registered to allow the saving to continue in the background.

If you would like recent changes to be made available to other devices, you can use `mergeWithCompletion:` instead of `processPendingChangesWithCompletion:`. The merge method also ensures changes are fully committed to disk before completing.

## Terminating on Mac

On the Mac, you should implement the `applicationShouldTerminate:` method to allow Ensembles to exit cleanly.

```

1 -(NSApplicationTerminateReply)applicationShouldTerminate:(NSApplication *)sender
2 {
3     [managedObjectContext save:NULL];
4
5     [persistentStoreEnsemble processPendingChangesWithCompletion:^(NSError *error) {
6         [[NSApplication sharedApplication] replyToApplicationShouldTerminate:YES];
7     }];
8
9     return NSTerminateLater;
10 }
```

Again, you can substitute `mergeWithCompletion:` for `processPendingChangesWithCompletion:` if you wish recent changes to be made available to other devices.

If the cloud service you are using needs to make use of the run loop for networking (*e.g.* Dropbox), you will need some extra steps. The `applicationShouldTerminate:` method is called with the main run loop in a modal mode. The following should suffice to keep the run loop turning over.

```

1 @implementation AppDelegate {
2     BOOL finishedTerminatingMerge;
3 }
4
5 ...
6
7 -(NSApplicationTerminateReply)applicationShouldTerminate:(NSApplication *)sender
8 {
9     [managedObjectContext save:NULL];
10
11     finishedTerminatingMerge = NO;
12     [persistentStoreEnsemble mergeWithCompletion:^(NSError *error) {
13         finishedTerminatingMerge = YES;
14         [[NSApplication sharedApplication] replyToApplicationShouldTerminate:YES];
15     }];
16
17     NSTimer *timer = [NSTimer timerWithTimeInterval:0.0 target:self
18         selector:@selector(waitForTerminatingMerge:) userInfo:nil repeats:NO];
19     [[NSRunLoop currentRunLoop] addTimer:timer forMode:NSModalPanelRunLoopMode];
20
21     return NSTerminateLater;
22 }
```

```
23
24 -(void)waitForTerminatingMerge:(NSTimer *)timer
25 {
26     while (!finishedTerminatingMerge) [[NSRunLoop currentRunLoop] runUntilDate:[NSDate date]];
27 }
28 }
```

## Delegating Saving

Some apps utilize a class or library to handle saves. This section includes a few popular examples, with guidance on how to deal with saving.

### UIManagedDocument

`UIManagedDocument` is a subclass of `UIDocument` available for use in iOS apps. It behaves like a document class, but uses a Core Data for storage. The class sets up the Core Data stack automatically, creating a main-thread child context, and a private-queue parent context for efficient saving.

`UIManagedDocument` changes the way saving is initiated. Instead of calling `save:` explicitly, you invoke the method `updateChangeCount:` with the argument `UIDocumentChangeDone`. The new data is not saved immediately; instead, the class will save after a few seconds, if no other changes are made.

If you have parts of your data model that are susceptible to the issues discussed above, you will want a way to force a synchronous save of the document data. You could add a method for this purpose to your `UIManagedDocument` subclass.

```
1 - (void)saveSynchronously
2 {
3     [self.managedObjectContext save:NULL];
4
5     [self.managedObjectContext.parentContext performBlockAndWait:^(
6         [self.managedObjectContext.parentContext save:NULL];
7     )];
8 }
```

When Ensembles merges changes, you need to make sure you merge the updates into both of the document contexts, as follows.

```

1 - (void)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble
2   didSaveMergeChangesWithNotification:(NSNotification *)notification
3 {
4     [self.managedObjectContext.parentContext performBlockAndWait:^(
5         [self.managedObjectContext.parentContext mergeChangesFromContextDidSaveNotificatio\
6 n:notification];
7     )];
8
9     [self.document.managedObjectContext performBlockAndWait:^(
10        [self.document.managedObjectContext mergeChangesFromContextDidSaveNotification:not\
11 ification];
12        )];
13 }

```

You should also be aware that asynchronous saves may get coalesced. Don't assume that if you call `updateChangeCount:` twice, that there will be two save operations. This could be important if you need to avoid the issues discussed in the section on [accidental duplicate](#). When in doubt, use synchronous saves to maintain control over what and when you commit to Ensembles.

## Magical Record

Magical Record also includes a background parent context that asynchronously saves to the persistent store. If you need to force synchronous saves, you can call `-MR_saveToPersistentStoreAndWait` on the default context.

# Models

## Loading a Model

When you first create a Core Data app, you often load your model using a convenience class method from `NSManagedObjectModel` like `+mergedModelFromBundles:`. Ensembles has its own internal Core Data model, so you have to be careful not to accidentally load the Ensembles model into your app's main `NSManagedObjectModel`.

In general, it is best to create a URL for your model, and load it explicitly using the `-initWithContentsOfURL:` initializer. If you do have multiple model files, you can merge them using `+modelByMergingModels:`.

```
1 NSURL *url1 = [[NSBundle mainBundle] URLForResource:@"Model1" withExtension:@"momd"];
2 NSManagedObjectModel *model1 = [[NSManagedObjectModel alloc] initWithContentsOfURL:url1];
3
4 NSURL *url2 = [[NSBundle mainBundle] URLForResource:@"Model2" withExtension:@"momd"];
5 NSManagedObjectModel *model2 = [[NSManagedObjectModel alloc] initWithContentsOfURL:url2];
6
7 NSManagedObjectModel *fullModel = [NSManagedObjectModel modelByMergingModels:@[model1, model2]];
8
```

## Designing a Model

TBW

- Try to avoid models that can become invalidated by concurrent changes
- Validation rules
- You should be careful what you put in custom setters and getters. They should not assume anything about the context they are used in, and should not have side effects. If you need more complex behavior, better to leave the setter/getter simple, and add extra methods (eg `changeX`)
- Orphaned objects
- Accumulating attributes. Use transactional objects instead.
- Including a unique id (or not). Global identity.
  - Different ways to create global ids
  - Tags, dates
  - Global identity versus repairs. Careful choice of global id can often avoid repairs such as orphaned objects.
- What if you have an existing app model, and want to move to ensembles?

## Excluding Entities and Properties

Usually you want the whole persistent store to be synced across devices, but sometimes you will have an entity or property that is only needed locally, and doesn't need to be transferred.

You could setup a separate store just for this local data, but that would be quite an extreme solution if you just have a few items to exclude from the sync.

Ensembles 2 allows you to have entities and properties ignored. Just add a value for the key *CDEIgnoredKey* to the user info section of your model for any entity or property (attribute or relationship). Set a non-zero integer as the value (eg "1") and the entity/property will be ignored by the framework.

## Migrations

TBW

- Only works with lightweight migrations
- Does not work with the new automatic migrations added in iOS 9. There must be a model version file for each version you ship. Ensembles uses those to determine if it has all the necessary model, and will not merge a file if it thinks it is missing a model version.
- Restriction: will not properly track name changes. Eg. entity name, or property name change
- Restriction: Making a property non-optional with a default won't work well, because the default is never saved, and thus never recorded in Ensembles
- If you need major manual migration, create a new ensemble (ie different ensemble identifier; eg MainStore.v2)
- How it works:
  - Ensembles records model used for each change set/event
  - If it identifies an unknown model, it gives merge error
  - It continues to store any changes, so there is no data loss
  - Will continue to give merge error until user updates to get new model
  - You can use the error code to inform user of the upgrade
  - When user upgrades, it merges everything, with no data loss

## Object Identity

One of the strengths of the Ensembles framework is how it handles object identity. You can control the global identifiers assigned to objects, which allows the framework to match objects from different devices. Used in the right way, this can be a powerful tool, and can help you avoid having to 'clean up' your data by deduplicating and merging objects.

### Global Identifiers

By default, the ensemble will generate random global identifiers for all objects. If logically-equivalent objects are inserted on multiple devices, after merging, there will be duplicates.



You can either deduplicate these objects in code, by fetching, and deleting a duplicate, or you can instead provide global identifiers which Ensembles can use to identify corresponding objects and merge them automatically.

Whenever the ensemble needs to know the global identifier of one or more objects, it will invoke this method. You can determine the identifiers as you please, but they must be immutable: an object should not have its global identifier change at any point. If you find you are tempted to change the global identifier of an object, it is better to delete the object instead, and create a new object with the new identifier.

The global identifiers do not have to be stored in the persistent store, but it often works out to be the best solution. You can either determine the global identifier from existing properties (eg email, tag), or store a random identifier like a uuid.

If you have certain objects in the array for which you do not wish to assign your own global identifier, you can return `NSNull` in that position.

```
1 - (NSArray *)persistentStoreEnsemble:(CDEPersistentStoreEnsemble *)ensemble globalIdentifi\
2 ersForManagedObjects:(NSArray *)objects;
```

## Introducing Global Identifiers to an Existing App

If you are introducing Ensembles to an existing app, you will likely want to migrate your data to include global identifiers. You should do this before leeching for the first time.

If your app was standalone to begin with, data will likely be unique to each device, and there should be no issue. Ensembles will simply merge the data from all devices.

If, on the other hand, your app had some other means of syncing prior to Ensembles, it may be possible that data is duplicated across devices. After merging, duplicates will appear to the user, which is clearly undesirable.

If you happen to have global identifiers in your data set prior to migrating to Ensembles, you can keep using these identifiers and the data should be properly deduplicated by the framework. But the more likely scenario is that you do not have any global identifiers. If this is the case, you have a number of options for making the transition without ending up with duplicate data:

1. Introduce global identifiers in the months leading up to the release that includes Ensembles. This should mean that many users will have updated and synced their data, including global identifiers, and when Ensembles is introduced it should merge without duplication.
2. Ask the user if they wish to merge data, or replace the local data with cloud data. If they elect to only keep the cloud data, delete the local store before leeching. (Backing up the local store prior to leeching might be a good idea.)
3. Try to form global identifiers from existing attributes. It may be possible to create identifiers that are nearly unique, and avoid most duplicates.

Let's consider the final case a bit further. If you are writing a note taking app, it may not be crazy to initially use some derivative of the text of a note as the unique identifier. For example, you could use the first 100

characters, or take a hash of the string. After the initial migration to Ensembles, new notes would just get a standard UUID.

For a note with the text ‘Hello there’, you could use the md5 hash (e8ea7a8d1e93e8764a84a0f3df4644de) as the identifier. If another device happens to have that note, it will merge the two notes as one (dedupe).

The likelihood of two unrelated notes having the same text should be reasonably low, and if they are merged by accident, it should not be a large loss of data. But you could reduce this risk even further by combining multiple attributes in the global identifier. For example, the name of the parent folder of the note, its creation or modification date, tags, and so forth. You should be able to come up with an identifier that is very unlikely to collide with an unrelated note.

Be aware that these *derived* global identifiers should only be generated once, and stored. They shouldn’t be calculated lazily from object attributes, because attributes can change over time, and global identifiers have to be immutable. After the initial migration to Ensembles, it is best to assign new objects global identifiers using random UUIDs and the like.

## Working with Data

### Deletions

- Why are deletions different?
- Deletions may be best handled as ‘soft deletions’ in some cases

### Managing External Files

For external files, like photos, you have two choices:

1. Include the files as binary `NSData` attributes in your Core Data model, with the external storage option turned on.
2. Store the files separately, and just store the filename as an attribute in the Core Data store.

For (1), Ensembles will automatically sync the data, and ensure that all data files are present before merging. The downside of this approach, and storing files as `NSData` blobs in general, is that most APIs act on files, so you have to export the blob to a temporary file in order to work with it.

For (2), Ensembles will sync the filenames, but you have to manage the files themselves yourself.

Even though Ensembles will not sync the files, you can still use it to make generic code for uploading and downloading the files. The Ensembles classes that conform to the `CDECloudFileSystem` protocol are used to access cloud data by the framework, but you can also use these classes directly yourself for querying and manipulating cloud files.

For example, the `CDEICloudFileSystem` class is used to upload and download data from iCloud. You can use this class directly to

1. Create a directory in the cloud for the files.

2. Get the contents of the directory.
3. Download any files that are in the cloud, but not on the local device.
4. Upload any files that are local but not in the cloud.

You also need to be mindful of the fact that changes to the Core Data store may not occur at the same time as the files are updated. Your app should gracefully handle the case that a file is not yet available to present, perhaps by displaying a placeholder image.

## Batched Traversals (Ensembles 2 only)

When Ensembles needs to work with data, it traverses your model's entities one at a time. For example, if it needs to apply changes from a different device to your local store, it will do one entity entirely, then the next, then the next, etc. Each time it adds objects for a new entity, it connects those objects up to the objects from the entities that it has already handled. When it gets to the last entity, the whole store should be updated.

By default this all happens in memory, without any saves. With a large data store, it is wise to try to break it up into smaller parts. That's where batched traversals comes in.

## Components of Batching

It has two parts:

1. You can stipulate the order that Ensembles will go through the entities.
2. You can stipulate that intermediate saves are allowed on certain entities before the whole model is handled, and how often these saves should occur.

## Ordering Entities

To stipulate the order, you enter key-value pairs in the User Info section of the data modeler.

1. Select an Entity in the Data Modeler.
2. Select the third tab in the Inspector on the right.
3. Click the + button in the *User Info* section.
4. Enter a positive integer value for the key `CDEMigrationPriorityKey`

**Entity**

Name:

Class:

☐ Abstract Entity

Parent Entity:

Indexes

| Key                      | Value |
|--------------------------|-------|
| CDEMigrationBatchSizeKey | 500   |
| CDEMigrationPriorityKey  | 20    |

**Versioning**

Hash Modifier:

Renaming ID:

Batch Settings

The higher the priority, the earlier the entity will be handled. The default priority is zero. If two entities have the same priority, they are simply ordered alphabetically.

The `CDEMigrationPriorityKey` allows you to handle certain entities first. Usually you will want to identify one or more subtrees of your model that can be handled and saved first. For example, one approach would be to migrate the leaf entities that contain lots of data first, and later connect the whole object graph to those leaves.

## Batch Saves

Ordering the entities would not be very useful for reducing memory usage if the framework cannot save data to disk. The `CDEMigrationBatchSizeKey` key can be used in an entity's user info for this purpose.

By default it is zero, which means no batches, and no saving, but if you set this to a positive integer on one of the entities, saves will occur after that number of objects have been processed. This also means any previously handled entities will also be saved, so you need to make sure that if you apply a batch size, that your object graph is able to be saved in an incomplete state. It places some restrictions on the entities you can choose for batching. Namely,

1. Any entity that has a positive batch size should not have any validation rules for relationships, as relationships may not be fully formed when saving.
2. Entities migrated prior to an entity with a positive batch size should not have validation rules for relationships to the batched entity, or even to entities that have a lower priority and have not been handled.

These restrictions make sense when you think about it. Just consider whether a save will succeed when your object graph is only partially built, and where it is acceptable to make 'cuts'. The cut points are the entities with batch sizes set.

The batch size itself will depend on the entity, and no doubt will require some guesswork, and profiling to see what keeps the memory usage at a reasonable level.

# Repairing Conflicts

TBW

- Concurrent Changes
  - Don't have to happen at exactly the same time
  - Could be separated in time.
  - Changes are concurrent when they are made in different stores, with no interceding merge
- Conflicts
  - Can occur when you have concurrent changes to the same property of corresponding objects
  - Which should be used?
  - Can have far reaching consequences? Eg relationships affect other objects
- How Changes are Applied
  - Ensembles replays changes in order. Latest wins.
  - Order is approximately time ordered, but may not be exact in all cases.
  - It uses a vector clock
  - Important is that all devices use same order
  - Ensembles does not check validity of object graph, so can become invalid due to conflicts
  - Ensembles has a series of delegate methods that are called to allow *reparation*
  - If you have a model that can become invalid due to conflicts, you should repair the model in one or more of these delegate methods. Shown in next section.
- Reparation
  - Ensemble Delegate Methods
  - Role of saving context
  - Role of reparation context
  - Preemptive repairs
  - Repairs to correct validation failures

# Testing and Debugging

## Unit Tests

Unit tests are included for the Ensembles framework on each platform. To run the tests, open the Xcode workspace, choose the Ensembles Mac or Ensembles iOS target in the toolbar at the top, and select the menu item `Product > Test`.

- Running unit and system tests
- Enabling logging
- Setting up a local file system backend
- Pasteboard backend

# Backends

This chapter introduces the various backends that Ensembles can work with. You will be introduced to the peculiarities of each, and how best to use them, including when it is best to attempt a merge.

## iCloud Drive

iCloud Drive is a file transfer backend built into all Apple devices. An attractive feature of iCloud Drive is that virtually all Apple customers have an iTunes account, and thereby an iCloud account, so you don't have to get them to sign up before they can sync with your app. The service is also free for the developer: customers get some free storage, and can pay for more as needed.

### Installing

You do not need to do anything to install the iCloud Drive cloud file system. It is included in the core functionality of Ensembles.

### Initializing

You should check whether the user of your app is logged into iCloud, and has the *Documents & Data* setting enabled, before trying to initialize an `CDEiCloudFileSystem` instance. To do the check, simply access the `NSFileManager` property `ubiquityIdentityToken`; if it is `nil`, the user is not logged in, or has *Documents & Data* switched off.

Once you know that the user is logged in and you have access to data storage in iCloud, you can initialize an `CDEiCloudFileSystem`.

```
1 CDEiCloudFileSystem *cloudFileSystem =  
2     [[CDEiCloudFileSystem alloc] initWithUbiquityContainerIdentifier:nil];
```

You pass the ubiquity container identifier as argument. This is usually your app identifier, prepended with 'iCloud', as in *iCloud.com.mentalfaculty.idiomatic*. Passing in `nil` for the container identifier will cause the app's default container to be used. For many apps, this will be adequate.

### Delegate Methods and Notifications

The `CDEiCloudFileSystem` class currently has no delegate methods, but it does fire the notification `CDEiCloudFileSystemDidDownloadFilesNotification` when it detects that one or more new files have finished downloading from the iCloud servers.



## When to Merge

In addition to merging on launch, termination, and entering the background, the firing of the `CDEICloud-FileSystemDidDownloadFilesNotification` notification offers a good opportunity to merge. It is fired when files from other devices have become available on the local device. Starting a merge operation will import these new files, and update the persistent store.

## CloudKit

CloudKit is a framework built into all Apple devices from iOS 8 and OS X 10.10 onwards. An attractive feature of CloudKit is that virtually all Apple customers have an iCloud account, so you don't have to get them to sign up before they can sync with your app. If you utilize CloudKit's private database, the service is also free for the developer: customers get some free storage, and can pay for more as needed.

## Installing

To use the CloudKit backend in your project, add the CloudKit framework to your target in Xcode, and then drag in the `CDECloudKitFileSystem.h` and `CDECloudKitFileSystem.m` files, which are in the Framework/Extensions directory of Ensembles.

If you are using Cocoapods, you simply need to add the CloudKit subspec to your Podfile.

```
1 pod "Ensembles/CloudKit", "~> 2.0"
```

This will link CloudKit, and include the `CDECloudKitFileSystem` class in your project.

You will also need to enable the CloudKit entitlement for your app, and choose an appropriate ubiquity identifier for your app. This will also require changes to your provisioning profiles to include CloudKit support.

## Initializing

You initialize an `CDECloudKitFileSystem` like this:

```
1 CDECloudKitFileSystem *cloudFileSystem =  
2     [[CDECloudKitFileSystem alloc]  
3         initWithPrivateDatabaseForUbiquityContainerIdentifier:@"iCloud.com.mentalfaculty.idiomat\  
4 ic"  
5         schemaVersion:CDECloudKitSchemaVersion2];
```

There is another initializer that allows use of the public database. The public database does not allow for zones, and Ensembles has to use a different technique for fetching data, which in our experience is not as robust or fast. If possible, use the initializer above, with the most recent schema.

You pass the ubiquity container identifier as argument. In this example, it is *iCloud.com.mentalfaculty.idiomat*. Do not pass in `nil` for the container identifier.

The schema parameter is for backwards compatibility. If you have a new app, use the most recent schema. Once you have chosen a schema, you should not generally change it, as this will require your users to start over with sync.

## Deploying

CloudKit enters a Development mode when you first add it to your app. You must switch it to Production mode in the CloudKit Dashboard before deploying the app in the store. Without that, your app will not sync.

When you deploy on CloudKit, you will be asked if you want to keep indexes that have not been used. It is easiest just to keep the indexes, but if you decide to remove them, thoroughly test the app's sync before shipping it.

## Delegate Methods and Notifications

There are currently no delegate methods or custom notifications, but you can use push notifications to trigger merges. This is discussed in the next section.

## When to Merge

CloudKit provides push notifications for when changes occur in the cloud. The `CDECloudKitFileSystem` class can setup a so-called subscription so that your app receives these push notifications, but there are still a few steps you need to take to handle them and trigger a merge.

You setup the CloudKit subscription when you create a new `CDECloudKitFileSystem` object.

```
1 CDECloudKitFileSystem *cloudFileSystem = [[CDECloudKitFileSystem alloc]
2 initWithPrivateDatabaseForUbiquityContainerIdentifier:@"iCloud.com.mentalfaculty.idiomatic\
3 "
4 schemaVersion:CDECloudKitSchemaVersion2];
5 [cloudFileSystem subscribeForPushNotificationsWithCompletion:^(NSError *error) {
6     if (error) NSLog(@"Failed to subscribe for push: %@", error);
7 }];
```

On iOS you need to register for remote notifications, usually in the `application:didFinishLaunchingWithOptions:` delegate method.

```
1 [[UIApplication sharedApplication] setMinimumBackgroundFetchInterval:UIApplicationBackgrou\
2 ndFetchIntervalMinimum];
3 [[UIApplication sharedApplication] registerForRemoteNotifications];
```

You then implement the `application:didReceiveRemoteNotification:` delegate method to trigger a merge.

```

1 - (void)application:(UIApplication *)application
2   didReceiveRemoteNotification:(NSDictionary *)userInfo
3   fetchCompletionHandler:(void (^)(UIBackgroundFetchResult))completionHandler
4 {
5     UIBackgroundTaskIdentifier backgroundIdentifier =
6         [[UIApplication sharedApplication] beginBackgroundTaskWithExpirationHandler:NULL];
7     [ensemble mergeWithCompletion:^(NSError *error) {
8         if (error) NSLog(@"Error: %@", error);
9         [[UIApplication sharedApplication] endBackgroundTask:backgroundIdentifier];
10        completionHandler(UIBackgroundFetchResultNewData);
11    }]];
12 }

```

It's also possible to setup background modes so that your app can retrieve and merge data when in the background. To do this, add the 'Remote notifications' background mode to the capabilities of your app target in Xcode.

On the Mac, you register for remote notifications in the `applicationDidFinishLaunching:` delegate method.

```

1 [[NSApplication sharedApplication] registerForRemoteNotificationTypes:NSRemoteNotification\
2  TypeNone];

```

Then implement the remote notification delegate methods to trigger a merge.

```

1 - (void)application:(NSApplication *)application
2   didRegisterForRemoteNotificationsWithDeviceToken:(NSData *)deviceToken
3 {
4     NSLog(@"%@ with token = %@", NSStringFromSelector(_cmd), deviceToken);
5 }
6
7 - (void)application:(NSApplication *)application
8   didFailToRegisterForRemoteNotificationsWithError:(NSError *)error
9 {
10    NSLog(@"%@ with error = %@", NSStringFromSelector(_cmd), error);
11 }
12
13
14 - (void)application:(NSApplication *)application
15   didReceiveRemoteNotification:(NSDictionary *)userInfo
16 {
17     [ensemble mergeWithCompletion:^(NSError *error){
18         if (error) NSLog(@"Error: %@", error);
19     }]];
20 }

```

## Dropbox Core API

[Dropbox](#)<sup>47</sup> is best known as a consumer file syncing service, but developers can also access file storage using various SDKs, including the REST-based Core API. There is an Objective-C SDK available to access this API from iOS and OS X. The Ensembles class `CDEDropboxCloudFileSystem` is built on the SDK. The class does not access files directly in the file system, even if they are accessible; instead, it makes HTTP requests to upload and download data.

### Signing Up

Before syncing your app via Dropbox, you need to sign up for a Dropbox Developer account, and register your app. Navigate to the [Dropbox Developer Site](#)<sup>48</sup>. Sign up for an account, then follow these steps:

1. In the App Console, click the *Create app* button.
2. Choose the *Dropbox API app* type.
3. Choose to store *Files and Datastores*.
4. Choose *Yes — My app only needs access to files it creates*.
5. Name the app.
6. Click on *Create app*.
7. Note the app key and secret.

### Installing

To manually install the Dropbox Core API backend, you must first install the SDK. If you are using Ensembles via Git, you can simply update the submodules to add the SDK to the *Vendor* folder.

```
1 git submodule update --init
```

You can also install the SDK by downloading it from the [Dropbox Developer Site](#)<sup>49</sup>, or by forking a repository on GitHub such as [this one](#)<sup>50</sup>.

Follow the instructions in the SDK to add it to your Xcode project.

Once you have added the Dropbox SDK to your project, you should drag in the `CDEDropboxCloudFileSystem.h` and `CDEDropboxCloudFileSystem.m` files, which are in the *Framework/Extensions* directory of Ensembles.

You will need to go into your the Build Settings of your target, and change the Header Search Path to include the Dropbox SDK directory. The path will depend on where the SDK is in relation to your Xcode project. You can construct a path relative to the `SRCROOT` variable, which is the directory containing your app's Xcode project. Here is what is used in the Idiomatic sample app:

---

<sup>47</sup><http://dropbox.com>

<sup>48</sup><http://developer.dropbox.com>

<sup>49</sup><https://www.dropbox.com/developers/>

<sup>50</sup><https://github.com/drewmccormack/dropbox-sdk-ios>

```
1 $(SRCROOT)/../../Vendor/DropboxSDK
```

It is important that you make this a *recursive* search path.

If you are using Cocoapods, you can ignore the information above, and simply add the Dropbox subspec to your Podfile.

```
1 pod "Ensembles/Dropbox", "~> 1.0"
```

This will link the Dropbox SDK, and include the `CEDropboxCloudFileSystem` class in your project.

## Initializing

To initialize the `CEDropboxCloudFileSystem`, you use the `initWithSession:` initializer. The argument is a `DBSession` instance, which you create using the Dropbox SDK.

```
1 DBSession *dropboxSession = [[DBSession alloc] initWithAppKey:@"<your app key>"
2     appSecret:@"<your app secret>" root:kDBRootAppFolder];
3 dropboxSession.delegate = self;
4
5 CEDropboxCloudFileSystem *dropboxSystem = [[CEDropboxCloudFileSystem alloc] initWithSess\
6     ion:dropboxSession];
7 dropboxSystem.delegate = self;
```

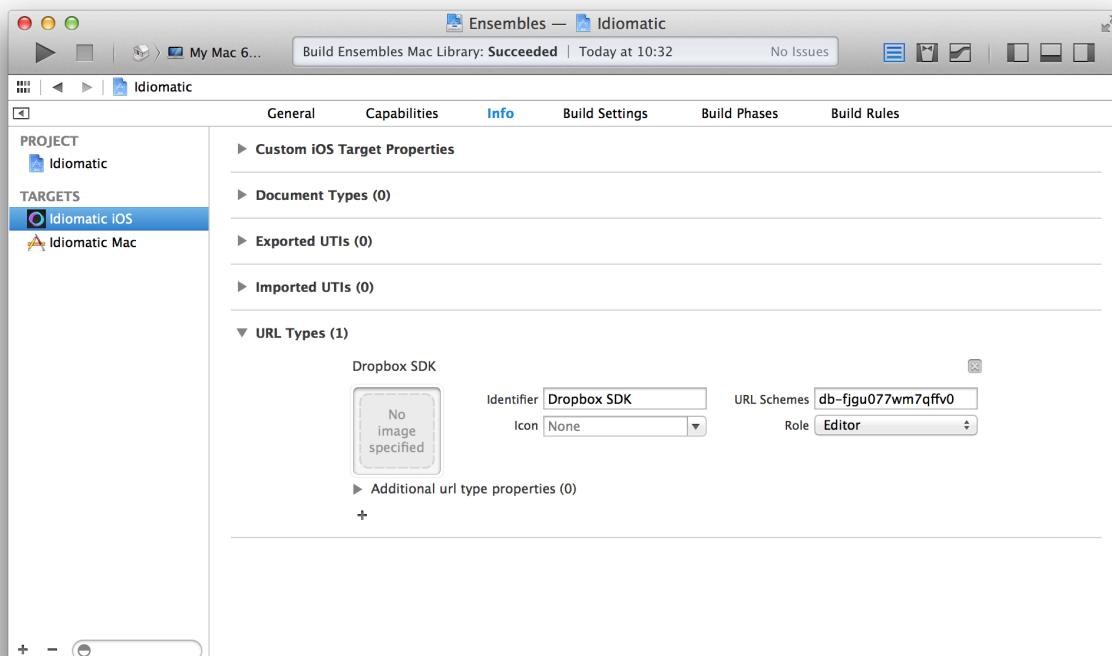
Enter the app key and secret you noted after registering your app at the Dropbox Developer Site.

The Dropbox API authenticates the user using OAuth. To authenticate, you present a view controller that the SDK provides. Once the user has authenticated, the SDK has to transfer control back to your app. It does this using a custom URL scheme. For this to work, you have to register your app to handle the URL scheme.

Select the Project root in the source list of Xcode, and then the App's target. Open the *Info* tab, and the URL Types section. Change the URL Schemes entry to

```
1 db-xxxxxxxxxxxxxx
```

Fill in your own app key in place of the x's. Finally, enter an identifier for the URL type (eg Dropbox SDK).



### Setting the Dropbox URL Scheme.

Since iOS 9, apps are required to register all URL schemes they will use in the `Info.plist` file. The Dropbox Core SDK makes use of some URL schemes for authentication, and these must be registered by your app.

To do this, open the `Info.plist` file of your app, and insert an entry for the key `LSApplicationQueriesSchemes`. The value should be an array, and include the string `dbapi-2`.

```

1 <key>LSApplicationQueriesSchemes</key>
2 <array>
3   <string>dbapi-2</string>
4 </array>

```

More info about the change can be found at [Dropbox](https://blogs.dropbox.com/developers/2015/08/important-update-your-core-api-app-for-ios-9/)<sup>51</sup>.

## Delegate Methods

In order for the Dropbox backend to operate, you need to have your controller class (eg App delegate) conform to the `CDEDropboxCloudFileSystemDelegate` protocol, and set the delegate of the `CDEDropboxCloudFileSystem` instance you created.

The required delegate method `-linkSessionForDropboxCloudFileSystem:completion:` is invoked when the `CDEDropboxCloudFileSystem` needs you to authenticate the user using the SDK. A typical authentication might look like this

<sup>51</sup><https://blogs.dropbox.com/developers/2015/08/important-update-your-core-api-app-for-ios-9/>

```

1 - (void)linkSessionForDropboxCloudFileSystem:(CDEDropboxCloudFileSystem *)fileSystem
2     completion:(CDECompletionBlock)completion
3 {
4     UIWindow *window = [[UIApplication sharedApplication] keyWindow];
5     dropboxLinkSessionCompletion = [completion copy];
6     [dropboxSession linkFromController:window.rootViewController];
7 }

```

This uses the DBSession method `linkFromController:` to present the login view. The completion handler has to be called back when the authentication is finished, to inform the CDEDropboxCloudFileSystem that it can continue, but because the authentication process is asynchronous, it is necessary to store the completion handler in an instance variable.

```

1 CDECompletionBlock dropboxLinkSessionCompletion;

```

You can call it when the app is requested to open the URL for the Dropbox scheme registered earlier.

```

1 - (BOOL)application:(UIApplication *)application handleOpenURL:(NSURL *)url
2 {
3     if ([dropboxSession handleOpenURL:url]) {
4         if ([dropboxSession isLinked]) {
5             if (dropboxLinkSessionCompletion) dropboxLinkSessionCompletion(nil);
6         }
7     } else {
8         NSError *error = [NSError errorWithDomain:CDEErrorDomain
9             code:CDEErrorCodeAuthenticationFailure userInfo:nil];
10        if (dropboxLinkSessionCompletion) dropboxLinkSessionCompletion(error);
11    }
12    dropboxLinkSessionCompletion = NULL;
13    return YES;
14 }
15 return NO;
16 }

```

If the DBSession is successfully linked, the completion callback is called with a `nil` argument; if an error occurred, an `NSError` instance is passed to the completion handler instead.

On Mac OS X you handle linking Dropbox differently; you can use the `DBAuthHelperOSX` class, which is included in Dropbox SDK.

```

1  @implementation AppDelegate
2
3  - (void)linkSessionForDropboxCloudFileSystem:(CDEDropboxCloudFileSystem *)fileSystem
4      completion:(CDECompletionBlock)completion
5  {
6      [DBSession setSharedSession:session];
7
8      __weak typeof(self) weakSelf = self;
9      id dropboxAuthHelperObserver =
10         [[NSNotificationCenter defaultCenter] addObserverForName:DBAuthHelperOSXStateChang\
11 edNotification
12         object:nil
13         queue:[NSOperationQueue mainQueue]
14         usingBlock:^(NSNotification *note) {
15             typeof(self) strongSelf = weakSelf;
16             NSError *error = note.userInfo[@"error"];
17             if (error) {
18                 if (completion) completion(error);
19                 [[NSNotificationCenter defaultCenter] removeObserver:dropboxAuthHelperObserver\
20 ];
21             }
22             else if (session.isLinked) {
23                 if (completion) completion(nil);
24                 [[NSNotificationCenter defaultCenter] removeObserver:dropboxAuthHelperObserver\
25 ];
26             }
27         }];
28     [[DBAuthHelperOSX sharedHelper] authenticate];
29 }
30
31 ...
32
33 @end

```

## When to Merge

The CDEDropboxCloudFileSystem class currently does not provide any notifications for cloud file updates, so you should just merge at standard times, such as launch, and when entering the background. You might also schedule an NSTimer to fire every few minutes and perform a merge.

## Deploying

Dropbox enters a Development mode when you first register your app. You must switch it to Production mode in the Developer Dashboard before deploying the app in the store.



## WebDAV

[WebDAV](http://en.wikipedia.org/wiki/WebDAV)<sup>52</sup> is cloud storage based on a HTTP standard, accessed as a web service. The Ensembles class `CDEWebDavCloudFileSystem` is built to access WebDAV services, and is included with some Ensembles packages in the Ensembles 2 framework.

### Service

To use this backend, you will need to have access to a WebDAV service. You can host your own, or subscribe to a WebDAV service.

### Installing

To use the WebDAV backend, drag in the `CDEWebDavCloudFileSystem.h` and `CDEWebDavCloudFileSystem.m` files, which are in the `Framework/Extensions` directory of Ensembles.

If you are using Cocoapods, you simply need to add the Dropbox subspec to your Podfile.

```
1 pod "Ensembles/WebDAV", "~> 2.0"
```

This will include the `CDEWebDavCloudFileSystem` class in your project.

### Initializing

To initialize `CDEWebDavCloudFileSystem`, you use the `initWithBaseURL:` initializer. The argument is a URL to the location that Ensembles should store its data. Note that the username for the WebDAV service is often included in the URL's path.

After initializing the file system, you should set the `username` and `password` properties. It is up to you how you have the user input these values, and where you store them. The keychain is a secure place for storage.

In addition to supplying login credentials, you should also set the `delegate`, in order to handle failed authentications, and possibly update the `baseURL` if needed.

### Delegate Methods

The delegate method `webDavCloudFileSystem:updateLoginCredentialsWithCompletion:` is called if authentication fails. You should request new credentials from the user and set the `username` and `password` properties, and then call the completion callback to indicate you are finished.

The `webDavCloudFileSystemWillFormURLRequest:` method is invoked just before a URL request is formed. It is useful if you need to update the `baseURL` because it is based on the `username` or other dynamic quantities. The URL of WebDAV services often includes the `username`, so the `baseURL` needs to be modified if a new `username` is adopted.

---

<sup>52</sup><http://en.wikipedia.org/wiki/WebDAV>

## When to Merge

The `CDEWebDavCloudFileSystem` class currently does not provide any notifications for cloud file updates, so you should just merge at standard times, such as launch, and when entering the background. You might also schedule an `NSTimer` to fire every few minutes and perform a merge.

## Zip Compression

The `CDEZipCloudFileSystem` is available only in Ensembles 2, and is not a standalone backend. It is used in combination with other backends to zip files before they are uploaded. For JSON transaction logs, this can lead to a dramatic reduction in file size.

## Installing

To use the Zip cloud file system, you must first install `SSZipArchive`. If you are using Ensembles via Git, you can simply update the submodules to add it to the *Vendor* folder.

```
1 git submodule update --init
```

It also available directly on GitHub [here](https://github.com/soffes/ssziparchive)<sup>53</sup>.

Once you have `SSZipArchive`, drag the source files into your project. Also drag in the `CDEZipCloudFileSystem.h` and `CDEZipCloudFileSystem.m` files, which are in the `Framework/Extensions` directory of Ensembles 2.

If you are using CocoaPods, you simply need to add the Zip subspec to your Podfile.

```
1 pod "Ensembles/Zip", "~> 2.0"
```

This will link `SSZipArchive`, and include the `CDEZipCloudFileSystem` class in your project.

## Initializing

To initialize the `CDEZipCloudFileSystem`, you use the `initWithCloudFileSystem:` initializer. The argument is the `CDECloudFileSystem` instance which is responsible for actually transferring the files to the cloud.

---

<sup>53</sup><https://github.com/soffes/ssziparchive>

```

1 CDEICloudFileSystem *iCloudFileSystem = [[CDEICloudFileSystem alloc] initWithUbiquityConta\
2   innerIdentifier:nil];
3 CDEZipCloudFileSystem *zipCloudFileSystem = [[CDEZipCloudFileSystem alloc] initWithCloudFi\
4   leSystem:iCloudFileSystem];
5 ensemble = [[CDEPersistentStoreEnsemble alloc] initWithEnsembleIdentifier:@"MainStore"
6   persistentStoreURL:storeURL
7   managedObjectModelURL:modelURL
8   cloudFileSystem:zipCloudFileSystem];

```

The Zip cloud file system is passed to the ensemble initializer, not the transferring file system.

## Encryption

The CDEEncryptedCloudFileSystem is available only in Ensembles 2, and is not a standalone backend. It is used in combination with other backends to encrypt files before they are uploaded. Files are encrypted using a symmetric AES key built upon some user provided password.

## Installing

To use the encrypted cloud file system, you must first install RNCryptor. If you are using Ensembles via Git, you can simply update the submodules to add it to the *Vendor* folder.

```
1 git submodule update --init
```

It is also available directly on GitHub [here](#)<sup>54</sup>.

Once you have RNCryptor, link to the target appropriate for your platform. Also link to the Security framework.

Drag in the CDEEncryptedCloudFileSystem.h and CDEEncryptedCloudFileSystem.m files, which are in the Framework/Extensions directory of Ensembles 2.

If you are using CocoaPods, you simply need to add the Encrypt subspec to your Podfile.

```
1 pod "Ensembles/Encrypt", "~> 2.0"
```

This will link RNCryptor, and include the CDEEncryptedCloudFileSystem class in your project.

## Initializing

To initialize the CDEEncryptedCloudFileSystem, you usually use the initWithCloudFileSystem:password: initializer. The arguments are the CDECloudFileSystem instance which is responsible for actually transferring the files to the cloud, and the password used to encrypt the data. (You should keep the password securely stored, perhaps in the keychain.)

---

<sup>54</sup><https://github.com/RNCryptor/RNCryptor>

```

1  NSString *password = @"<Retrieve password from Keychain>";
2  CDEICloudFileSystem *iCloudFileSystem =
3      [[CDEICloudFileSystem alloc] initWithUbiquityContainerIdentifier:nil];
4  CDEEncryptedCloudFileSystem *encryptedCloudFileSystem =
5      [[CDEEncryptedCloudFileSystem alloc] initWithCloudFileSystem:iCloudFileSystem
6          password:password];
7  ensemble = [[CDEPersistentStoreEnsemble alloc] initWithEnsembleIdentifier:@"MainStore"
8      persistentStoreURL:storeURL
9      managedObjectModelURL:modelURL
10     cloudFileSystem:encryptedCloudFileSystem];

```

The encrypted cloud file system is passed to the ensemble initializer, rather than the transferring file system.

## Ensembles Server (Node.js, S3)

Ensembles Server is a basic Node.js web app that utilizes Amazon's S3 for file storage, and a Postgres database for user management. It can be used with the `CDENodeCloudFileSystem` in the Ensembles framework on iOS devices. The server offers HTTP Basic Authentication, and an API that supports user actions such as signing up, logging in, changing password, and resetting a password (emails new password).

### Installing

See the section on [Deploying Ensembles Server](#).

### Initializing

To initialize the `CDENodeCloudFileSystem` instance, you need to provide a username and password.

```

1  NSURL *url = [NSURL URLWithString:@"https://<your app>.herokuapp.com"];
2  NSString *username = [[NSUserDefaults standardUserDefaults] valueForKey:IDMNodeS3EmailDefaultKey] ? : @"";
3  NSString *password = [self retrievePassword];
4  CDENodeCloudFileSystem *newNodeFileSystem = [[CDENodeCloudFileSystem alloc] initWithBaseURL:url];
5  newNodeFileSystem.delegate = self;
6  newNodeFileSystem.username = username;
7  newNodeFileSystem.password = password;
8  newSystem = newNodeFileSystem;

```

You can store the username and password however you please, but the Keychain is probably the most secure place to store authentication credentials.

## Delegate Methods

The required delegate method `-nodeCloudFileSystem:updateLoginCredentialsWithCompletion:` is invoked by the `CDENodeCloudFileSystem` when it needs to authenticate with Ensembles Server. You need to present a request to the user to enter the username and password, set the corresponding properties on the `CDENodeCloudFileSystem` object, and then call the completion block.

In this particular implementation, the completion block is stored in an instance variable (`nodeCredentialUpdateCompletion`), and a view controller is presented.

```

1 - (void)nodeCloudFileSystem:(CDENodeCloudFileSystem *)fileSystem
2   updateLoginCredentialsWithCompletion:(CDECompletionBlock)completion
3 {
4     [self clearPassword];
5     nodeCredentialUpdateCompletion = [completion copy];
6
7     // Present the node settings view
8     UIWindow *window = [[UIApplication sharedApplication] keyWindow];
9     UIStoryboard *storyboard = [UIStoryboard storyboardWithName:@"Main" bundle:nil];
10    UINavigationController *nodeSettingsNavController =
11        [storyboard instantiateViewControllerWithIdentifier:@"NodeSettingsNavigationContro\
12    ller"];
13    IDMNNodeSyncSettingsViewController *settingsController =
14        (id)nodeSettingsNavController.topViewController;
15    settingsController.nodeFileSystem = fileSystem;
16
17    [window.rootViewController presentViewController:nodeSettingsNavController
18        animated:YES completion:NULL];
19 }

```

The loaded view controller calls the `storeNodeCredentials` method when the user enters a username and password. This method sets the username and password on the `CDENodeCloudFileSystem` object, and also stores the values for future use. It calls the stored completion callback, passing `nil` upon success, and an `NSError` if something goes wrong.

```

1 - (void)storeNodeCredentials
2 {
3     CDENodeCloudFileSystem *nodeFileSystem = (id)self.ensemble.cloudFileSystem;
4     NSString *email = nodeFileSystem.username;
5     NSString *password = nodeFileSystem.password;
6     NSError *error = nil;
7     if (email && password) {
8         [[NSUserDefaults standardUserDefaults] setObject:email
9             forKey:IDMNNodeS3EmailDefaultKey];
10        [self storePassword:password];

```

```

11     }
12     else {
13         NSDictionary *info =
14             @{NSLocalizedStringKey : @"Invalid username or password"};
15         NSError *error = [NSError errorWithDomain:CDEErrorDomain
16                             code:CDEErrorCodeAuthenticationFailure userInfo:info];
17     }
18
19     if (nodeCredentialUpdateCompletion) nodeCredentialUpdateCompletion(error);
20     nodeCredentialUpdateCompletion = NULL;
21 }

```

The view controller also offers the opportunity for the user to cancel the login. If this happens, the `cancelNodeCredentialsUpdate` is called instead, and the completion handler is called with an error.

```

1 - (void)cancelNodeCredentialsUpdate
2 {
3     NSError *error = [NSError errorWithDomain:CDEErrorDomain
4                         code:CDEErrorCodeCancelled userInfo:nil];
5     if (nodeCredentialUpdateCompletion)
6         nodeCredentialUpdateCompletion(error);
7     nodeCredentialUpdateCompletion = NULL;
8 }

```

## Storing Passwords in the Keychain

The Keychain is probably the safest place to store the user's password. Because it has a plain C API, the Keychain can be intimidating at first.

Objective-C wrappers classes which make it more palatable are available. A popular choice is [SSKeychain](https://github.com/soffes/sskeychain)<sup>55</sup>.

If you would rather use the C API directly, link the *Security* framework to your app, and adapt the following methods.

```

1 - (NSDictionary *)keychainQuery {
2     NSString *serviceName = @"<Your service name as reverse DNS>";
3     return @{
4         (__bridge id)kSecClass : (__bridge id)kSecClassGenericPassword,
5         (__bridge id)kSecAttrService : serviceName,
6         (__bridge id)kSecAttrAccount : serviceName,
7         (__bridge id)kSecAttrAccessible : (__bridge id)kSecAttrAccessibleAlways
8     };
9 }
10

```

---

<sup>55</sup><https://github.com/soffes/sskeychain>

```

11 - (void)storePassword:(NSString *)newPassword
12 {
13     NSMutableDictionary *keychainQuery = [[self keychainQuery] mutableCopy];
14     SecItemDelete((__bridge CFDictionaryRef)keychainQuery);
15     keychainQuery[(__bridge id)kSecValueData] =
16         [newPassword dataUsingEncoding:NSUTF8StringEncoding];
17     SecItemAdd((__bridge CFDictionaryRef)keychainQuery, NULL);
18 }
19
20 - (NSString *)retrievePassword
21 {
22     NSMutableDictionary *keychainQuery = [[self keychainQuery] mutableCopy];
23     keychainQuery[(__bridge id)kSecReturnData] = @YES;
24     keychainQuery[(__bridge id)kSecMatchLimit] = (__bridge id)kSecMatchLimitOne;
25
26     NSString *result = nil;
27     CFDataRef data = NULL;
28     if (noErr == SecItemCopyMatching((__bridge CFDictionaryRef)keychainQuery,
29         (CTypeRef *)&data)) {
30         result = [[NSString alloc] initWithData:(__bridge id)data
31             encoding:NSUTF8StringEncoding];
32     }
33     if (data) CFRelease(data);
34
35     return result;
36 }
37
38 - (void)clearPassword
39 {
40     NSDictionary *keychainQuery = [self keychainQuery];
41     SecItemDelete((__bridge CFDictionaryRef)keychainQuery);
42 }

```

You need to set the service name to something unique, usually in reverse DNS form.

## When to Merge

The `CDNodeCloudFileSystem` class currently does not provide any notifications for cloud file updates, so you should just merge at standard times, such as launch, and when entering the background. You might also schedule an `NSTimer` to fire every few minutes and perform a merge.

# Adding a Custom Backend

## CDECloudFileSystem

TBW